优化算法

第六周：从 Neural Tangent Kernel 到 μP
——跨模型的超参数迁移

讲义

2025 年春季学期

本周核心问题

- 为什么在小模型上调好的超参数，直接用到大模型上往往失败？
- 训练过程中，网络到底在 “学特征” 还是在 “做核回归”？
- 是否存在一种参数化方式，使得最优超参数**与模型宽度无关**？

教学提示

本周教学节奏（两次课，每次 ~100 分钟）：

第一次课：NTK（手算 1–5 + 练习 1–4）

- 0–20min：手算 1（单神经元 NTK，完整 6 步）
- 20–25min：练习 1（换数据）
- 25–50min：一般定义 + 两层网络 NTK 推导（手算 2，完整展开）
- 50–70min：手算 3（$m = 4$ 个 ReLU 神经元，学生自己算）+ 练习 2
- 70–80min：Jacot 三大定理 + 手算 4（量级分析）
- 80–95min：核回归 + 手算 5（闭式预测）+ 练习 3（提前停止）
- 95–100min：NTK 的致命缺陷（留悬念）

第二次课：μP（手算 6–10 + 练习 5–8）

- 0–15min：动机 + μP 定义
- 15–35min：abc-参数化推导（手算 6，完整展开）
- 35–60min：核心对比手算 7（NTK vs μP，分组并排计算）+ 练习 5
- 60–75min：超参数迁移定理 + 手算 8（学习率曲线）+ 练习 6
- 75–90min：Transformer 的 μP（手算 9，注意力缩放）+ 手算 10（完整梯度下降对比）
- 90–100min：全景图 + 开放问题 + 作业布置

全程叙事线：**为什么你在 A100 上调好的学习率，换到 H200 集群上跑大模型就炸了？**

目录

第零节：大模型训练的现实 **5**

I 第一次课：Neural Tangent Kernel **13**

1 PyTorch 默认做了什么？——标准参数化（SP） **13**

1.1 Kaiming 初始化 13

1.2 SP 的学习率：所有层一视同仁 13

1.3 SP 的核心问题：最优 η^* 随宽度漂移 14

1.4 SP vs μP：预览 14

2 深度学习理论的三大困难——为什么需要 NTK？ **14**

2.1 困难 1：非凸性 15
2.2 困难 2：过参数化 15
2.3 困难 3：高维性 15
2.4 NTK 的突破：绕过所有三个困难 15

3 热身：一个最小的例子 17

4 NTK 的一般定义 20
4.1 从参数空间到函数空间：Jacobian 矩阵 21
4.2 函数空间的精确演化方程 21

5 两层网络的 NTK：完整推导 23
5.1 设置与符号 23
5.2 对每个参数求梯度 23

6 L 层网络的 NTK 递归 27

7 Jacot 三大定理与核冻结 33

8 核回归与谱分析 36

9 NTK 的致命缺陷：丢失 Feature Learning 37
9.1 什么是 Feature Learning? 37
9.2 NTK 为什么没有 Feature Learning? 38
9.3 Feature Learning 值多少? 39

II 第二次课：μP 与超参数迁移 40

10 μP 的定义与动机 40
10.1 NTK vs μP：工程写法的主要差别 40

11 μP 的推导：从前向传播到特征变化 40
11.1 模型假设 40
11.2 约束 1：前向传播稳定 41
11.3 约束 2：损失增量 ΔL 不随 d 爆炸 41
11.4 约束 3：特征变化 ΔY_k 稳定（μP 的关键约束） 42
11.5 SGD 版本的 μP 规则汇总 43
11.6 Adam 版本（实践中更常用） 44
11.7 工程 μP 写法 46

12 NTK vs μP：训练时间尺度的本质区别 46

13 超参数迁移 48

14 Transformer 的 μP 修改 **50**
14.1 完整修改清单 51
14.2 μP 在视觉模型上的进展 51

15 Tensor Programs 框架（概述） **53**

16 参数化族全景图 **53**

17 局限与开放问题 **53**

18 扩展阅读：Mean Field 理论视角 **54**
18.1 从粒子系统的角度看神经网络 54
18.2 $m \to \infty$：从粒子到分布 54
18.3 三种极限的关系 55
18.4 Wasserstein 距离与特征学习的量化 55
18.5 NTK 作为 Mean Field 的线性化 56

19 本周总结 **56**

第零节：大模型训练的现实——为什么超参数迁移重要？

教学提示

这一节是**动机铺垫**，用 10 分钟让学生对大模型训练的“体量”有具体感知。很多学生只在单卡上跑过小实验，对“一次训练花几百万美元”没有概念。用具体数字建立直觉后，后面讲 μP 的实用价值时学生才会觉得“这确实是个大问题”。

主流模型到底多大？

（一）大语言模型（LLM）

模型	年份	参数量	层数 L	d_{model}	训练数据量
GPT-2	2019	1.5B	48	1600	40GB 文本
GPT-3	2020	175B	96	12288	570GB（300B tokens）
LLaMA-2-70B	2023	70B	80	8192	2T tokens
LLaMA-3-405B	2024	405B	126	16384	15T tokens
DeepSeek-V3	2024	671B (MoE)	61	7168	14.8T tokens

参数量粗略估计（decoder-only Transformer）：$P \approx 12\,L\,d_{\text{model}}^2$。每层 $\approx 12d^2$（attention 的 W_Q, W_K, W_V, W_O 共 $4d^2$，FFN 共 $8d^2$）。验证：LLaMA-2-70B：$12 \times 80 \times 8192^2 \approx 64B$，接近 70B。✓

数据量的量纲感：“token”到底是什么？

训练数据量用“token”计量。Token **不是**单词，而是经过 **BPE（Byte-Pair Encoding）** 分词后的子词单元。BPE 的核心思想是：从单个字符开始，反复合并出现频率最高的相邻字符对，直到词表达到预设大小（通常 32K–128K 个 token）。

原始文本	BPE 分词结果	token 数
“Hello world”	[“Hello”, “␣world”]	2
“unbelievable”	[“un”, “believ”, “able”]	3
“深度学习”	[“深度”, “学习”] 或 [“深”, “度”, “学”, “习”]	2–4
“transformers”	[“transform”, “ers”]	2
“###ERROR 404”	[“###”, “ERROR”, “␣404”]	3

经验换算：

- 英文：1 token ≈ 0.75 个单词 ≈ 4 个字符。“A nice day”=3 个词 ≈3–4 个 token。
- 中文：1 个汉字 ≈1–2 个 token（取决于词表）。中文文本的 token 数通常比英文多 30–50%。

- 代码：变量名和关键字通常是 1 个 token，但长变量名会被拆分。

大数字的直觉：

Token 数	≈	等价于
1B（10^9）	≈	4GB 原始文本 ≈ 4000 本书
1T（10^{12}）	≈	4TB 原始文本 ≈ 400 万本书 ≈ 英文维基百科的 250 倍
15T	≈	60TB ≈ 整个公开互联网文本的多次遍历

LLaMA-3 在 15T tokens 上训练，相当于把互联网“读了十几遍”。注意：不同模型用不同的 BPE 词表（LLaMA 用 SentencePiece，GPT 用 tiktoken），所以**不同模型的“1T tokens”对应的原始文本量略有差异**，但量级一致。

（提问学生：为什么不直接用字符或单词作为 token？——字符太细粒度，序列太长，attention 复杂度 $O(s^2)$ 爆炸。单词词表太大（英文 >100 万个词形），且无法处理新词。BPE 是中间方案：词表小（∼32K–128K），覆盖率高，序列长度适中。）

（二）图像生成模型（Diffusion / Flow Matching）

模型	年份	参数量	架构	训练数据
Stable Diffusion 1.5	2022	0.9B	UNet	LAION-5B（50 亿图文对）
SDXL	2023	3.5B	UNet	内部数据集
Stable Diffusion 3.5	2024	8B	MMDiT	内部数据集
DALL·E 3	2023	未公开	UNet	内部数据 + 合成 caption
FLUX.1	2024	12B	MMDiT	未公开

趋势：从 UNet（∼1B）到 Diffusion Transformer/MMDiT（∼8–12B），参数量增长了 ∼10 倍。架构从 CNN 转向了 Transformer——**与 LLM 用同样的骨架**，所以 μP 的参数化理论同样适用。

（三）视频生成模型

模型	年份	参数量（估）	训练数据	训练成本（估）
Sora (v1)	2024	未公开（∼3–10B?）	未公开	4K–10K 张 H100×1 月
Open-Sora 2.0	2025	∼1–3B	30M 视频片段（80K 小时）	**$200K**（公开）
Movie Gen (Meta)	2024	30B	未公开	6144 张 GPU

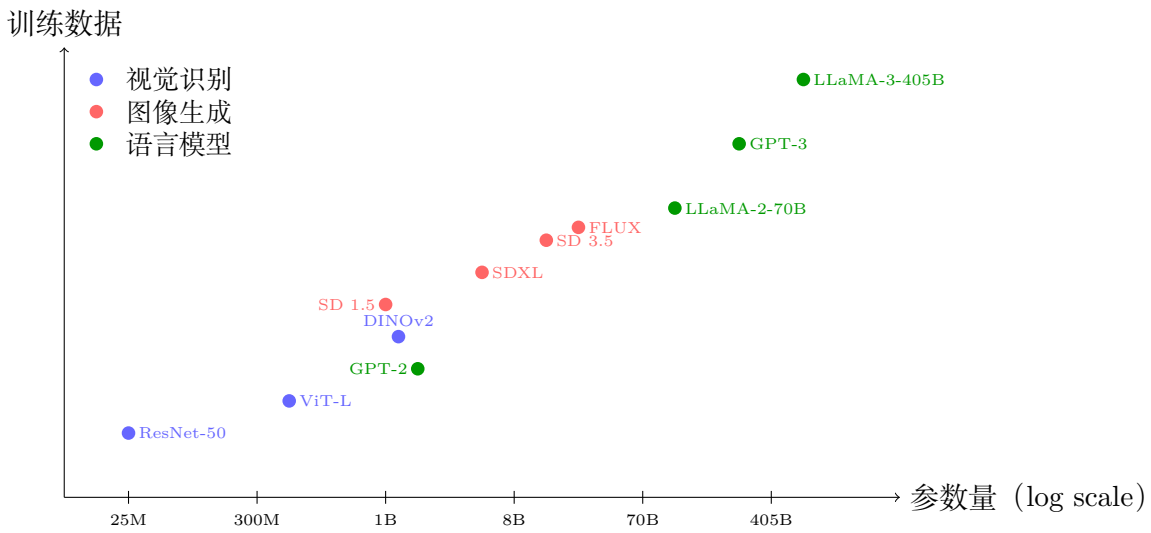
视频生成的计算量远超图像：每帧是一张图，加上帧间的时序 attention。生成 10 秒视频的推理成本 ≈ 几千次 ChatGPT 查询。

（四）视觉识别模型（对比用）

模型	年份	参数量	训练数据
ResNet-50	2015	25M	ImageNet（1.2M 张图，1000 类）
ViT-Large	2020	307M	ImageNet/JFT-300M
DINOv2-giant	2023	1.1B	LVD-142M（1.42 亿张图）
CLIP ViT-L/14	2021	428M	WIT（4 亿图文对）

（提问学生：你平时跑的模型多大？——可能是 ResNet-50(25M)或 BERT-base(110M)。GPT-3 是 BERT 的 1600 倍，FLUX 是 ResNet-50 的 480 倍。这就是为什么“在小模型上调好的超参数直接迁移”是一个价值几百万美元的问题。）

（五）全景对比：参数量 vs 训练数据量

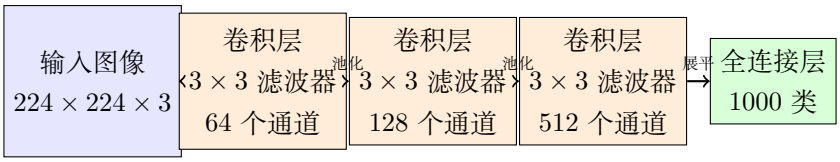


关键观察：所有领域都在沿“参数量↑，数据量↑”的方向 scaling。Diffusion 模型正在经历 LLM 三年前走过的路——从 ∼1B 到 ∼10B 再到更大。**μP 的超参数迁移在每个领域的 scaling 过程中都有价值。**

这些模型的内部长什么样？——CNN vs Transformer

本周的理论（NTK、μP）对全连接网络推导，但实际应用在 CNN 和 Transformer 上。我们先快速过一遍它们的核心区别，这样后面讲“层宽 m”“隐藏维度 d_{model}”时学生才知道对应的是什么。

（一）CNN（卷积神经网络）——图像的经典架构

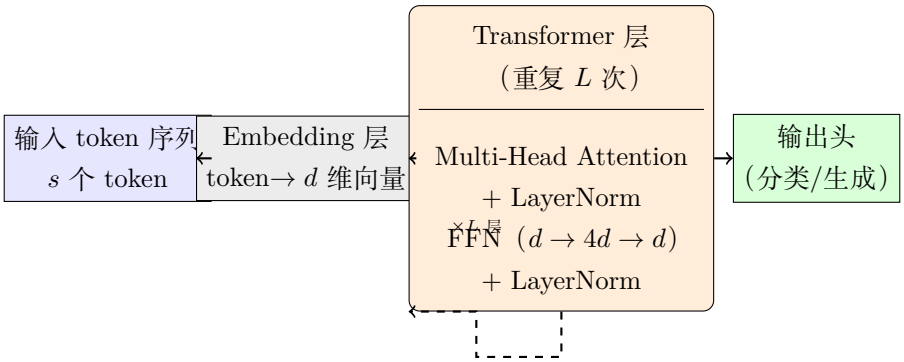


核心组件：

- **卷积层：**一组小滤波器（如 3×3）在图像上"滑动"，每个滤波器检测一种局部模式（边缘、纹理、角点等）。**参数共享**：同一个滤波器在所有位置重复使用，所以参数量很少（$3\times3\times C_{\text{in}}\times C_{\text{out}}$）。
- **池化层：**缩小空间分辨率（如 $224\to112\to56\to\cdots$），同时增加通道数（$64\to128\to256\to512$）。
- **感受野：**每个神经元只"看到"输入的一个局部区域。深层神经元的感受野更大——通过逐层堆叠实现"从局部到全局"。

参数量在哪？ ResNet-50 的 25M 参数中，∼90% 在卷积层（滤波器权重），∼10% 在最后的全连接层。"宽度"对应通道数 C。

（二）Transformer——语言和视觉的统一架构



每个 Transformer 层内部有两个核心子层：

(a) Multi-Head Self-Attention（自注意力）

每个 token"看"所有其他 token，计算"该关注谁"：

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}}\right)V.$$

- $Q=XW_Q$，$K=XW_K$，$V=XW_V$：三个线性投影，$W_Q,W_K,W_V\in\mathbb{R}^{d\times d}$。
- $QK^\top\in\mathbb{R}^{s\times s}$：注意力矩阵，第 (i,j) 元素表示 token i"关注"token j 的程度。
- softmax 归一化后乘以 V：加权汇聚信息。
- **Multi-head**：把 d 维切成 h 个 $d_{\text{head}}=d/h$ 维的"头"，每个头学不同的注意力模式，最后拼起来通过 $W_O\in\mathbb{R}^{d\times d}$ 投影回 d 维。
- 参数量：W_Q,W_K,W_V,W_O 共 $4d^2$。

（b）Feed-Forward Network（FFN，逐位置前馈网络）

$$\mathrm{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2, \qquad W_1 \in \mathbb{R}^{d\times 4d},\ W_2 \in \mathbb{R}^{4d\times d}.$$

- 先升维到 $4d$（扩展），过激活函数（ReLU 或 GeLU），再降维回 d。
- 对每个 token **独立**应用（不像 attention 那样看其他 token）。
- 参数量：$d \times 4d + 4d \times d = 8d^2$。

  每层总参数 $\approx 4d^2 + 8d^2 = 12d^2$。这就是前面 $P \approx 12Ld^2$ 的来源。

（三）关键区别对比

	CNN	**Transformer**
基本操作	局部卷积（3×3 滑窗）	全局注意力（每个 token 看所有 token）
参数共享	空间共享（同一滤波器全图用）	无空间共享（但跨 token 共享权重）
感受野	逐层扩大（局部 → 全局）	每层就是全局
“宽度”	通道数 C	隐藏维度 d_{model}
“深度”	卷积层数	Transformer 层数 L
序列/空间建模	靠层叠加（间接）	靠 attention 矩阵（直接）
计算复杂度	$O(s \cdot C^2 \cdot k^2)$（k= 核大小）	$O(s^2 \cdot d)$（attention）$+O(s \cdot d^2)$（FFN）
主要瓶颈	大分辨率图像	长序列（s^2）

关键直觉——为什么 μP 对 Transformer 特别重要

Transformer 的“宽度”就是 d_{model}。当我们把模型从 $d = 256$（代理模型）扩展到 $d = 8192$（大模型）时：

- 参数量增长 $\sim (8192/256)^2 = 1024$ 倍。
- 每层 attention 的 $QK^\top$ 矩阵大小变了，softmax 的“锐度”变了。
- FFN 的中间维度从 $4 \times 256 = 1024$ 变到 $4 \times 8192 = 32768$。
- 如果用标准参数化，最优学习率会随 d 系统性漂移。μP 通过修改输出缩放和注意力缩放（$1/\sqrt{d} \to 1/d$），让最优超参数**不随 d 变化**。

CNN 的“宽度”是通道数 C。理论上 μP 也适用（输出缩放 $1/C$，隐藏层学习率 η/C），但实践中 CNN 的 C 通常较小（≤ 2048），finite-width 效应更显著，μP 的收益没有 Transformer 那么明显。

训练一次要多少钱？

领域	模型	GPU	训练时长	估算成本
语言	GPT-3 175B	∼1K 张 V100	∼1 个月	\$4–12M
	LLaMA-2-70B	2K 张 A100	∼25 天	\$2–5M
	LLaMA-3-405B	16K 张 H100	∼54 天	\$30–60M
图像	SD 1.5	256 张 A100	∼1 个月	∼\$600K
	FLUX 12B	未公开	未公开	估 ∼\$2–5M
视频	Sora (v1)	4K–10K 张 H100	∼1 个月	估 \$10–30M
	Open-Sora 2.0	公开	公开	**\$200K**（开源标杆）

关键观察：

- 视频生成的训练成本可比 LLM（时空 attention 的计算量远超纯文本）。
- Open-Sora 2.0 用 \$200K 达到了接近商业水平——说明**工程优化和超参数选择**的重要性。
- 如果最优学习率调错了 2 倍，整个训练可能要重来——再花几百万美元。

这就是 μP 的实际价值：在小代理模型（单卡几小时）上调好超参数，直接迁移到大模型，**省掉大模型上的网格搜索**。

GPU 显存里装了什么？

训练大模型时 GPU 显存是最紧张的资源。以一个 P 参数的模型为例（混合精度训练，Adam 优化器）：

组件	大小	典型占比
模型参数（FP16）	$2P$ 字节	∼8%
梯度（FP16）	$2P$ 字节	∼8%
Adam 状态（FP32 的 m_t 和 v_t）	$2 \times 4P = 8P$ 字节	∼33%
FP32 主参数副本	$4P$ 字节	∼17%
激活值（用于反向传播）	**与 batch size 成正比**	∼34%

总计：**每个参数 ≈16–20 字节**（不含激活值）。

代入具体数字： 7B 参数模型，纯参数 + 优化器状态 $\approx 7 \times 10^9 \times 16\text{B} = 112\text{GB}$。一张 A100 只有 80GB 显存——**单卡连 7B 的优化器状态都放不下**，必须用模型并行（如 DeepSpeed ZeRO）把状态切分到多张卡上。

Batch size 为什么重要？

激活值显存 $\propto$ batch size $\times$ 序列长度 $\times$ 隐藏维度 $\times$ 层数。

粗略公式（Transformer，无 activation checkpointing）：

$$\text{激活值显存} \approx 2 \times L \times B \times s \times d_{\text{model}} \times 10 \text{ 字节}$$

（B=batch size，s= 序列长度，系数 10 来自 attention logits、中间层输出等）。

数值例子： LLaMA-7B（$L = 32, d = 4096, s = 2048$）：

- $B = 1$：激活值 $\approx 2 \times 32 \times 1 \times 2048 \times 4096 \times 10 \approx 5$GB。可以。
- $B = 32$：≈ 160GB。单卡放不下！
- $B = 1024$（大规模训练的典型值）：≈ 5TB。需要几十张卡。

Activation checkpointing（梯度检查点）可以用计算换显存：只保存每隔几层的激活值，其余在反向传播时重新计算。典型可节省 60–70% 激活值显存，代价是 ~30% 额外计算。

关键直觉——为什么 batch size 缩放是个大问题

实际大模型训练的 batch size 通常很大（$B = 512$ 到 $B = 4096$ 甚至更大），因为：

- 大 batch 利用 GPU 并行度，**吞吐量高**（tokens/秒）。
- 梯度方差 $\propto 1/B$，大 batch 的梯度估计更准。
- 但大 batch 需要**调大学习率**（线性缩放法则 $\eta \propto B$），否则收敛变慢。

问题是：**μP 保证学习率跨宽度可迁移，但不直接处理 batch size 缩放**。从小 batch 调好的 η 到大 batch 不能直接用——需要额外的 scaling law（如 $\eta \propto \sqrt{B}$ 或 $\eta \propto B$）。这是 μP 的一个重要局限，也是活跃的研究方向。

训练 vs 推理的成本对比

	训练	推理（生命周期内）
计算量（FLOPs）	$\sim 6PD$（一次性）	取决于用户量，可远超训练
典型成本占比	10–30%	**70–90%**
谁付钱	模型开发者	最终通过 API 定价转嫁

其中 P= 参数量，D= 训练 token 数。$6PD$ 的因子 6 来自：前向 $2PD$（每个参数每个 token 做一次乘加 = 2 次 FLOP），反向 $4PD$（约为前向的 2 倍）。

数值例子： LLaMA-2-70B 在 2T tokens 上训练：$6 \times 70 \times 10^9 \times 2 \times 10^{12} = 8.4 \times 10^{23}$ FLOPs。H100 峰值约 10^{15} FLOPS（FP16），理论最短时间 $= 8.4 \times 10^{23}/10^{15} = 8.4 \times 10^8$ 秒 ≈ 27 年（单卡）。用 2000 张卡：~5 天（假设利用率 50%）。

推理方面： GPT-4 级别的模型，如果每天服务 1 亿次请求、每次生成 500 个 token，每天的推理计算量就可能超过一次完整训练的计算量。所以**长期看推理成本远高于训练**——但训练成本是“不可重复的沉没成本”，超参数调错了就直接打水漂，这就是 μP 的价值。

小结：为什么本周内容重要

- 大模型训练**一次几百万美元**，超参数调错 = 重来 = 再花几百万。
- 显存限制使得 batch size、模型宽度的选择相互耦合。
- μP 提供了**跨宽度的超参数迁移**：在小模型上几小时调好，大模型直接用。
- 但 batch size 和深度方向的迁移**还不能**用 μP 解决——这是开放问题。

Part I

第一次课：Neural Tangent Kernel

1 PyTorch 默认做了什么？——标准参数化（SP）

在讲 NTK 之前，我们先搞清楚一个基本问题：**当你写 `nn.Linear(m_in, m_out)` 时，PyTorch 到底做了什么？**

1.1 Kaiming 初始化

PyTorch 的 `nn.Linear` 默认使用 Kaiming uniform 初始化（He et al., 2015）：

$$W_{ij} \sim \text{Uniform}\left(-\frac{1}{\sqrt{m_{\text{in}}}}, \frac{1}{\sqrt{m_{\text{in}}}}\right), \qquad \text{方差} = \frac{1}{3m_{\text{in}}}.$$

为什么除以 $\sqrt{m_{\text{in}}}$？考虑一层线性变换 $y = Wx$，$W \in \mathbb{R}^{m_{\text{out}} \times m_{\text{in}}}$。
$y_i = \sum_{j=1}^{m_{\text{in}}} W_{ij} x_j$。如果 x_j 的方差为 σ_x^2，W_{ij} 的方差为 σ_W^2，则

$$\text{Var}[y_i] = m_{\text{in}} \cdot \sigma_W^2 \cdot \sigma_x^2.$$

要让 $\text{Var}[y_i] = \text{Var}[x_j]$（信号方差逐层不变），需要 $\sigma_W^2 = 1/m_{\text{in}}$。

对 ReLU 激活，由于 $\text{Var}[\text{ReLU}(z)] = \frac{1}{2}\text{Var}[z]$（正半边保留一半方差），Kaiming init 修正为 $\sigma_W^2 = 2/m_{\text{in}}$。

★ 手算 0：验证 PyTorch 的初始化（课堂快速演示，3 分钟）

用 Python 验证：

```
import torch.nn as nn
layer = nn.Linear(1024, 512)  # m_in=1024, m_out=512
print(layer.weight.std())     # 约 0.018   1/sqrt(3*1024)

# 验证输出方差
x = torch.randn(100, 1024)    # 输入方差=1
y = layer(x)
print(y.std())                 # 约 1.0（信号幅度保持）
```

要点：Kaiming init 保证**前向传播时信号不爆炸不消失**。但它对**训练过程中**各层的更新量级没有任何保证——不同层的梯度量级可能差很远。

1.2 SP 的学习率：所有层一视同仁

PyTorch 的 `Adam(model.parameters(), lr=1e-3)` 对所有参数使用**同一个学习率**。这意味着：

- 嵌入层（$d_{\text{vocab}} \times d_{\text{model}}$）、隐藏层（$d \times d$）、输出层（$d \times d_{\text{vocab}}$）全用同一个 η。
- 但各层的梯度量级不同！输出层直接接 loss，梯度较大；深层的隐藏层梯度经过多次链式法则，可能偏小。
- 结果：有些层“学太快”（更新过大），有些层“学太慢”（更新过小）。

1.3 SP 的核心问题：最优 η^* 随宽度漂移

关键直觉——SP 为什么不支持超参数迁移

在 SP 下，当宽度 m 变化时：

- 初始化方差 $\sigma_W^2 = 1/m$ 保证了前向信号稳定。
- 但梯度的量级和 m 的关系**没有被控制**。
- 例如：隐藏层梯度 $\nabla_W L$ 中包含 $1/\sqrt{m}$ 因子（来自前向缩放），但求和时有 m 项，两者部分抵消但不完全。
- 最终效果：最优学习率 $\eta^*(m)$ 随 m 变化——通常 m 越大 η^* 越大（因为每步的有效更新变小了，需要更大的 η 补偿）。

这就是 μP 要解决的问题。μP 不是改初始化（Kaiming 已经够好），而是改**输出缩放**和**逐层学习率**，使得各层的更新量级在任何宽度 m 下都匹配。

1.4 SP vs μP：预览

后面我们会看到，μP 只需要在 SP 的基础上做**两处修改**：

	SP（PyTorch 默认）	μP
初始化	Kaiming: $\sigma_W^2 = 1/m$	不变（或 $\sigma_W^2 = 1$ 配合前向缩放 $1/\sqrt{m}$）
输出层缩放	$1/\sqrt{m}$	$\mathbf{1/m}$
隐藏层学习率	η（统一）	$\eta/\mathbf{m}$（Adam 下；SGD 下不缩放）
输出层学习率	η	η
最优 η^*	随 m 漂移	**不依赖** m

但要理解**为什么**这两处修改就够了，我们需要先理解 NTK 理论（第 2–6 节），然后从 abc-参数化框架推导出 μP（第二次课）。

2 深度学习理论的三大困难——为什么需要 NTK？

上一节讲了实际问题（超参数不可迁移），但 NTK 的诞生不是为了解决这个工程问题。NTK 回答的是一个更根本的**理论问题**：

深度学习理论的核心追问

为什么梯度下降在神经网络上能 work？
这个问题看起来简单，但经典优化理论和统计学给出的答案全是“不应该 work”。

2.1 困难 1：非凸性

神经网络的损失函数 $L(\theta)$ 关于参数 θ 是**高度非凸**的。即便是两层 ReLU 网络，损失曲面也充满鞍点和局部极小。

经典优化理论说：梯度下降只保证收敛到**驻点**（梯度为零的点），不保证收敛到全局最小。非凸问题一般是 NP-hard 的。

但实际上：SGD 几乎总能把训练误差降到接近零，甚至在没有正则化的情况下。

2.2 困难 2：过参数化

现代网络的参数量 P 远超训练样本量 n（$P \gg n$）。GPT-3 有 175B 参数，但训练数据即使有 300B tokens 也只有有限的独立信息。

经典统计学说：$P \gg n$ 应该导致严重**过拟合**——模型记住了训练数据但在测试集上表现很差。偏差-方差权衡预测存在一个最优的 P，过大会变差。

但实际上：更大的模型不仅训练误差更低，**测试误差也更低**（“double descent” 现象）。过参数化似乎是有益的。

2.3 困难 3：高维性

参数空间的维度 P 可达 10^{12}。在如此高维的空间里分析梯度流的全局行为，传统工具力不从心。

2.4 NTK 的突破：绕过所有三个困难

Jacot, Gabriel & Hongler (2018) 的 NTK 理论给出了一个**一举三得**的答案：

核心思路：不在 P 维参数空间里分析，而是**直接在 n 维函数空间里写出训练动力学**。当宽度 $m \to \infty$ 时：

1. **解决非凸性：**函数空间的动力学 $\dot{\mathbf{u}} = -\mathbf{\Theta}^*(\mathbf{u} - \mathbf{y})$ 是**线性 ODE**。线性系统没有局部极小，全局收敛到 $\mathbf{u} = \mathbf{y}$。**参数空间的非凸性在函数空间中“消失”了**。

2. **解决过参数化：**$m \to \infty$ 意味着 $P \to \infty$，这是“极端”过参数化。但 infinite-width 网络等价于 NTK 核回归——一个经典的、理论完善的学习算法。过参数化不但不有害，反而**使问题可解**（$\mathbf{\Theta}^*$ 正定保证了唯一解的存在）。

3. **绕过高维性：**虽然参数空间是 P 维的，但训练动力学可以完全用 $n \times n$ 的 NTK 矩阵 $\mathbf{\Theta}^*$ 描述。从 $P \sim 10^{12}$ 维降到 $n \sim 10^4$ 维——降维了 10^8 倍。

备注 2.1: 重要的细微处：深度 L 的角色

上述论证对**任意有限深度** L 成立——只要宽度 $m \to \infty$ 先取极限，L 固定。函数空间的动力学永远是线性 ODE，形式上永远 "凸"。

但**深度越大，Θ^* 的条件越差**：

对无 skip connection 的 ReLU 全连接网络，NTK 矩阵的条件数

$$\kappa(L) = \frac{\lambda_{\max}(\Theta^*)}{\lambda_{\min}(\Theta^*)} \xrightarrow{L\to\infty} \infty.$$

为什么？深层网络的 NNGP 核相关系数 $\rho^{(l)}$ 逐层递增趋向 1——所有输入在经过很多层非线性后变得 "越来越像"。当 $\rho^{(l)} \to 1$ 时，NTK 矩阵趋向**秩 1**（只有一个大特征值，其余趋于零），网络失去了区分不同输入的能力。

后果：收敛保证 $L(t) \le L(0)e^{-\eta\lambda_{\min}t}$ 形式上成立，但 $\lambda_{\min} \to 0$ 意味着收敛**无限慢**——"凸但没用"。

深度 L	$\kappa(\Theta^*)$（典型值）	收敛速度
2	~ 4	快
10	~ 100	慢
50	$\sim 10^6$	极慢
$\to \infty$	$\to \infty$	不收敛

ResNet 的 skip connection 部分缓解了这个退化：skip connection 相当于在 $\rho^{(l)}$ 的递推中加了一个 "拉回 $\rho < 1$" 的项，防止核坍缩到秩 1。这是 ResNet 优于 plain network 的 **NTK 层面的解释**。

宽度 m 和深度 L 同时 $\to \infty$ 呢？ NTK 理论的标准证明要求 $m \to \infty$ 时 L 固定。如果 m, L 同时增长（如实际的 GPT-3：$L = 96, d = 12288$），需要更精细的分析——这是 Tensor Programs VI (Yang et al., 2024) 的内容，也是 μP 在深度方向迁移困难的理论原因。

关键直觉——NTK 的理论地位

NTK 理论第一次给出了**在严格数学意义上**，神经网络梯度下降全局收敛的证明。它不是一个近似论证或启发式直觉，而是一个完整的**定理**：在 m 足够大的条件下，梯度下降以指数速率收敛到零训练误差。

这是深度学习理论的里程碑。在 NTK 之前，"为什么 GD 能 work" 这个问题基本是 open 的。NTK 给出了第一个完整的答案——虽然这个答案有代价（丢失 feature learning），但它建立了整个后续理论的**概念框架**：核冻结 vs 核漂移、lazy vs rich training、线性化 vs 非线性动力学。μP 理论正是在这个框架上发展起来的。

本节后续路线：先用最小的例子手算 NTK（第 3 节），然后给出一般定义和两层网络的完整推导（第 4 节），再陈述 Jacot 三大定理（第 5 节），最后讨论 NTK 的代价——丢失 feature

learning（第 8 节）。这为第二次课的 μP 做好铺垫。

3 热身：一个最小的例子

在进入 NTK 理论之前，我们先手算一个最小的网络。目标：让每个概念都有**具体的数字**支撑。

★ 手算 1：单神经元 Sigmoid 的 NTK（课堂演示，约 20 分钟）

设置。最简单的 “网络”：单个 sigmoid 神经元，标量输入 $d = 1$。

$$f(x; a, w) = a\,\sigma(wx), \qquad \sigma(z) = \frac{1}{1+e^{-z}}.$$

参数 $\theta = (a, w) \in \mathbb{R}^2$，初始化 $a_0 = 1, w_0 = 0$。
训练数据两个点：$(x_1, y_1) = (1, 1)$ 和 $(x_2, y_2) = (-1, 0)$。
MSE 损失：$L = \frac{1}{2}[(f(x_1) - y_1)^2 + (f(x_2) - y_2)^2]$。

第 1 步：计算初始输出。
先算 sigmoid 在 0 处的值：

$$\sigma(0) = \frac{1}{1+e^0} = \frac{1}{2} = 0.5.$$

因此

$$f(x_1; \theta_0) = a_0 \cdot \sigma(w_0 \cdot x_1) = 1 \times \sigma(0 \times 1) = 1 \times 0.5 = 0.5,$$

$$f(x_2; \theta_0) = a_0 \cdot \sigma(w_0 \cdot x_2) = 1 \times \sigma(0 \times (-1)) = 1 \times 0.5 = 0.5.$$

输出向量：$\mathbf{u}(0) = (f(x_1), f(x_2))^\top = (0.5, 0.5)^\top$。
目标向量：$\mathbf{y} = (y_1, y_2)^\top = (1, 0)^\top$。
初始残差：$\mathbf{r}(0) = \mathbf{u}(0) - \mathbf{y} = (0.5 - 1,\ 0.5 - 0)^\top = (-0.5,\ 0.5)^\top$。
初始损失：$L(0) = \frac{1}{2}((-0.5)^2 + 0.5^2) = \frac{1}{2} \times 0.5 = 0.25$。
（板书：在坐标轴上画出 x 轴为输入，y 轴为输出，标出 $(1, 0.5)$ 和 $(-1, 0.5)$ 两个当前输出点，用虚线标出目标 $(1, 1)$ 和 $(-1, 0)$。）

第 2 步：计算所有偏导数。
我们需要 f 对每个参数的偏导。先准备 sigmoid 的导数公式：

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

在 $z = 0$ 处：$\sigma'(0) = 0.5 \times (1 - 0.5) = 0.5 \times 0.5 = 0.25$。

对参数 a 的偏导：
$f(x; a, w) = a\sigma(wx)$，所以 $\frac{\partial f}{\partial a} = \sigma(wx)$。
在 $x = x_1 = 1$ 处：$\frac{\partial f}{\partial a}\big|_{x_1} = \sigma(w_0 \cdot 1) = \sigma(0) = 0.5$。
在 $x = x_2 = -1$ 处：$\frac{\partial f}{\partial a}\big|_{x_2} = \sigma(w_0 \cdot (-1)) = \sigma(0) = 0.5$。

对参数 w 的偏导：

$\frac{\partial f}{\partial w} = a \cdot \sigma'(wx) \cdot x$（链式法则）。
在 $x = x_1 = 1$ 处：$\frac{\partial f}{\partial w}\Big|_{x_1} = 1 \times 0.25 \times 1 = 0.25$。
在 $x = x_2 = -1$ 处：$\frac{\partial f}{\partial w}\Big|_{x_2} = 1 \times 0.25 \times (-1) = -0.25$。
（提问学生：为什么 $\partial f/\partial w$ 在两个点上符号相反？——因为 $x_1 = 1$ 而 $x_2 = -1$。增大 w 会让 $f(x_1)$ 变大但让 $f(x_2)$ 变小。这正是 w 控制“形状”的体现。）

第 3 步：写出 Jacobian 矩阵。
Jacobian 矩阵 $\mathbf{J} \in \mathbb{R}^{n \times P} = \mathbb{R}^{2\times 2}$，第 i 行是 $\nabla_\theta f(x_i)^\top$：

$$\mathbf{J} = \begin{pmatrix} \frac{\partial f}{\partial a}\Big|_{x_1} & \frac{\partial f}{\partial w}\Big|_{x_1} \\ \frac{\partial f}{\partial a}\Big|_{x_2} & \frac{\partial f}{\partial w}\Big|_{x_2} \end{pmatrix} = \begin{pmatrix} 0.5 & 0.25 \\ 0.5 & -0.25 \end{pmatrix}.$$

观察：第一列（对 a 的梯度）两行相同——因为 a 对两个输出的影响方式相同（都是乘以 σ，而 $\sigma(0)$ 对称）。第二列（对 w 的梯度）两行符号相反——w 对两个输出的影响是**反向**的。

第 4 步：组装 NTK 矩阵。
NTK 矩阵 $\mathbf{\Theta} = \mathbf{J}\mathbf{J}^\top \in \mathbb{R}^{2\times 2}$。逐元素计算：
Θ_{11}（x_1 和 x_1 之间）：

$$\Theta_{11} = \nabla_\theta f(x_1)^\top \nabla_\theta f(x_1) = \left(\frac{\partial f}{\partial a}\bigg|_{x_1}\right)^2 + \left(\frac{\partial f}{\partial w}\bigg|_{x_1}\right)^2 = 0.5^2 + 0.25^2 = 0.25 + 0.0625 = 0.3125.$$

Θ_{12}（x_1 和 x_2 之间）：

$$\Theta_{12} = \nabla_\theta f(x_1)^\top \nabla_\theta f(x_2) = 0.5 \times 0.5 + 0.25 \times (-0.25) = 0.25 - 0.0625 = 0.1875.$$

Θ_{22}（x_2 和 x_2 之间）：

$$\Theta_{22} = 0.5^2 + (-0.25)^2 = 0.25 + 0.0625 = 0.3125.$$

合起来：

$$\boxed{\mathbf{\Theta} = \begin{pmatrix} 0.3125 & 0.1875 \\ 0.1875 & 0.3125 \end{pmatrix} = \frac{1}{16}\begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}.}$$

（提问：$\mathbf{\Theta}$ 是对称正定的。为什么？——因为 $\mathbf{\Theta} = \mathbf{J}\mathbf{J}^\top$，而 $\mathbf{J}$ 是 2×2 且行列式 $= 0.5 \times (-0.25) - 0.5 \times 0.25 = -0.25 \neq 0$，所以 $\mathbf{J}$ 满秩，$\mathbf{\Theta}$ 正定。）
物理意义：

- $\Theta_{11} = \Theta_{22} = 0.3125$：两个训练点的“自核值”相等（因为网络对 x_1, x_2 的梯度范数相同）。
- $\Theta_{12} = 0.1875 > 0$：训练 x_1 时，x_2 的输出也会**同方向**变化（正耦合）。

第 5 步：特征分解。

对 2×2 对称矩阵 $\begin{pmatrix}\alpha & \beta\\ \beta & \alpha\end{pmatrix}$，特征值为 $\alpha\pm\beta$，特征向量为 $(1,\pm 1)/\sqrt{2}$。

代入 $\alpha=5/16,\beta=3/16$：

$$\lambda_1=\frac{5+3}{16}=\frac{8}{16}=\frac{1}{2}=0.5,\qquad \mathbf{v}_1=\frac{1}{\sqrt{2}}\begin{pmatrix}1\\1\end{pmatrix},\tag{1}$$

$$\lambda_2=\frac{5-3}{16}=\frac{2}{16}=\frac{1}{8}=0.125,\qquad \mathbf{v}_2=\frac{1}{\sqrt{2}}\begin{pmatrix}1\\-1\end{pmatrix}.\tag{2}$$

条件数：$\kappa=\lambda_1/\lambda_2=0.5/0.125=4$。

验证：$\mathbf{\Theta}=0.5\cdot\mathbf{v}_1\mathbf{v}_1^\top+0.125\cdot\mathbf{v}_2\mathbf{v}_2^\top=0.5\cdot\frac{1}{2}\begin{pmatrix}1&1\\1&1\end{pmatrix}+0.125\cdot\frac{1}{2}\begin{pmatrix}1&-1\\-1&1\end{pmatrix}=\begin{pmatrix}0.3125&0.1875\\0.1875&0.3125\end{pmatrix}$。✓

（板书：画出 $\mathbb{R}^2$ 平面，标出 $\mathbf{v}_1$ 和 $\mathbf{v}_2$ 方向，在 $\mathbf{v}_1$ 方向画粗箭头标“快（$\lambda_1=0.5$）”，在 $\mathbf{v}_2$ 方向画细箭头标“慢（$\lambda_2=0.125$）”。）

第 6 步：预测训练动力学。

如果 NTK 冻结（$m\to\infty$ 的结论，这里 $m=1$ 只是近似），则

$$\mathbf{r}(t)=e^{-\mathbf{\Theta}t}\mathbf{r}(0).$$

将 $\mathbf{r}(0)=(-0.5,0.5)^\top$ 分解到特征基：

$$c_1=\mathbf{v}_1^\top\mathbf{r}(0)=\frac{1}{\sqrt{2}}(1\times(-0.5)+1\times 0.5)=0,\tag{3}$$

$$c_2=\mathbf{v}_2^\top\mathbf{r}(0)=\frac{1}{\sqrt{2}}(1\times(-0.5)+(-1)\times 0.5)=\frac{-1}{\sqrt{2}}\approx -0.707.\tag{4}$$

所以 $\mathbf{r}(0)=0\cdot\mathbf{v}_1+\frac{-1}{\sqrt{2}}\mathbf{v}_2$。**残差全在慢方向 $\mathbf{v}_2$ 上！**

动力学：

$$\mathbf{r}(t)=c_1e^{-\lambda_1 t}\mathbf{v}_1+c_2e^{-\lambda_2 t}\mathbf{v}_2=\frac{-1}{\sqrt{2}}e^{-t/8}\mathbf{v}_2=e^{-t/8}\begin{pmatrix}-0.5\\0.5\end{pmatrix}.$$

回到原始坐标：$\mathbf{u}(t)=\mathbf{y}+\mathbf{r}(t)=\begin{pmatrix}1\\0\end{pmatrix}+e^{-t/8}\begin{pmatrix}-0.5\\0.5\end{pmatrix}=\begin{pmatrix}1-0.5e^{-t/8}\\0.5e^{-t/8}\end{pmatrix}$。

损失：$L(t)=\frac{1}{2}\|\mathbf{r}(t)\|^2=\frac{1}{2}\times 2\times 0.25\times e^{-t/4}=0.25\,e^{-t/4}$。

收敛时间常数：$\tau=4$（损失半衰期 $t_{1/2}=4\ln 2\approx 2.77$）。

数值表（板书）：

t	0	4	8	16	32
$L(t)$	0.250	0.092	0.034	0.005	10^{-4}
$f(x_1)$	0.500	0.816	0.932	0.987	0.999
$f(x_2)$	0.500	0.184	0.068	0.013	0.001

第 7 步：几何解读（最重要！停下来讨论 2 分钟）

$\mathbf{v}_1 = (1,1)/\sqrt{2}$ 方向的含义：两个输出**同增同减**。哪个参数能做到？——调 a！增大 a 同时放大 $f(x_1)$ 和 $f(x_2)$。NTK 告诉我们这个方向"快"（$\lambda_1 = 0.5$）。

$\mathbf{v}_2 = (1,-1)/\sqrt{2}$ 方向的含义：两个输出**一增一减**。哪个参数能做到？——调 w！增大 w 让 sigmoid 变陡，$f(1)\uparrow$ 但 $f(-1)\downarrow$。NTK 告诉我们这个方向"慢"（$\lambda_2 = 0.125$），因为 sigmoid 在 $w = 0$ 附近变化很平缓（$\sigma'(0) = 0.25$ 只有最大值的一半）。

我们的学习任务需要 $f(x_1)$ 增大同时 $f(x_2)$ 减小——恰好**全在慢方向上**。如果任务换成 $f(x_1)$ 和 $f(x_2)$ 都增大，就全在快方向上了——这正是下面练习 1 的内容。

▷ 课堂练习 1：换一组标签（5 分钟）

用同一个网络和初始化，但把标签改为 $y_1 = 1, y_2 = 1$。

(a) NTK 矩阵 $\mathbf{\Theta}$ 是否变化?

(b) 新的残差 $\mathbf{r}(0) =$?

(c) $\mathbf{r}(0)$ 在哪个特征方向上?

(d) 收敛时间常数 $\tau =$?，和手算 1 比快了多少倍?

详细答案： (a) 不变。NTK$= \mathbf{J}\mathbf{J}^\top$ 只依赖输入 x_i 和当前参数 θ，不依赖标签 y_i。(b) $\mathbf{r}(0) = (0.5-1, 0.5-1)^\top = (-0.5,-0.5)^\top$。(c) $c_1 = \mathbf{v}_1^\top \mathbf{r}(0) = \frac{1}{\sqrt{2}}(-0.5-0.5) = -1/\sqrt{2}$，$c_2 = \mathbf{v}_2^\top \mathbf{r}(0) = \frac{1}{\sqrt{2}}(-0.5+0.5) = 0$。全在**快方向** $\mathbf{v}_1$ 上！(d) $\tau = 1/\lambda_1 \times 4 = 2$（损失 $L(t) = 0.25e^{-t}$），比手算 1 快 **4 倍**（$\kappa = 4$）。

结论：同一个 NTK，不同任务难度可以差 κ 倍。NTK 的条件数直接控制了"最难学的模式比最易学的模式慢多少"。

4 NTK 的一般定义

定义 4.1: Neural Tangent Kernel

对参数化函数 $f : \mathbb{R}^d \times \mathbb{R}^P \to \mathbb{R}$，在参数 θ 处的 NTK 定义为

$$\Theta_\theta(x, x') = \langle \nabla_\theta f(x;\theta), \nabla_\theta f(x';\theta)\rangle_{\mathbb{R}^P} = \sum_{p=1}^{P} \frac{\partial f(x)}{\partial \theta_p}\frac{\partial f(x')}{\partial \theta_p}.$$

NTK 矩阵 $\mathbf{\Theta} \in \mathbb{R}^{n\times n}$ 定义为 $\Theta_{ij} = \Theta(x_i, x_j)$，即 $\mathbf{\Theta} = \mathbf{J}\mathbf{J}^\top$。

关键直觉

NTK$\Theta(x, x')$ 度量的是："想让 $f(x)$ 变大" 和 "想让 $f(x')$ 变大" 这两个目标在参数空间里**有多对齐**。

$\Theta(x, x')$ 大 $\Rightarrow$ 训练 x 时 x' 也会 "搭便车" 变化。

$\Theta(x, x') = 0 \Rightarrow$ 两者完全解耦。

$\Theta(x, x') < 0 \Rightarrow$ 训练 x 时 x' 会**反方向**变化（互相干扰）。

4.1 从参数空间到函数空间：Jacobian 矩阵

我们的目标是：不追踪 P 个参数的运动，而是**直接**写出 n 个训练点上输出的演化方程。连接这两个空间的桥梁就是 Jacobian 矩阵。

定义 Jacobian 矩阵。网络 $f(x;\theta)$ 对**所有参数** $\theta \in \mathbb{R}^P$ 的梯度是一个 P 维向量 $\nabla_\theta f(x;\theta)$。对 n 个训练点分别求梯度，排成矩阵的行：

$$\mathbf{J}(t) = \begin{pmatrix} \nabla_\theta f(x_1;\theta(t))^\top \\ \nabla_\theta f(x_2;\theta(t))^\top \\ \vdots \\ \nabla_\theta f(x_n;\theta(t))^\top \end{pmatrix} \in \mathbb{R}^{n\times P}.$$

写开来，第 (i,p) 元素就是 $J_{ip} = \frac{\partial f(x_i)}{\partial \theta_p}$。

(回忆手算 1： $n = 2$ 个训练点，$P = 2$ 个参数 (a, w)，Jacobian 就是 2×2 矩阵 $\mathbf{J} = \begin{pmatrix} 0.5 & 0.25 \\ 0.5 & -0.25 \end{pmatrix}$。**)**

Jacobian 的两个方向。

$\mathbf{J}$ 把参数空间的运动映射到函数空间：如果参数变化 $\Delta\theta \in \mathbb{R}^P$，输出的变化为

$$\Delta\mathbf{u} \approx \mathbf{J}\,\Delta\theta \in \mathbb{R}^n.$$

（一阶近似。$\mathbf{J}$ 的每一行告诉你 "θ 动一下，$f(x_i)$ 变多少"。）

$\mathbf{J}^\top$ 把函数空间的 "信号" 映射回参数空间：如果有一个 "每个训练点该走多少" 的向量 $\mathbf{v} \in \mathbb{R}^n$，对参数的影响为

$$\mathbf{J}^\top\mathbf{v} = \sum_{i=1}^{n} v_i\,\nabla_\theta f(x_i) \in \mathbb{R}^P.$$

（$\mathbf{J}^\top$ 的每一列是一个训练点对参数的 "拉力"。）

4.2 函数空间的精确演化方程

定理 4.1: 训练动力学方程（精确，无近似）

对 MSE 损失 $L(\theta) = \frac{1}{2}\|\mathbf{u}(\theta) - \mathbf{y}\|^2$，连续时间梯度流下：

$$\boxed{\dot{\mathbf{u}}(t) = -\boldsymbol{\Theta}(t)\,(\mathbf{u}(t) - \mathbf{y}), \qquad \boldsymbol{\Theta}(t) = \mathbf{J}(t)\mathbf{J}(t)^\top.}$$

推导（逐步展开）：

第 1 步：参数空间的梯度流。梯度下降 $\dot{\theta} = -\nabla_\theta L$。对 MSE 损失：

$$\frac{\partial L}{\partial \theta_p} = \sum_{i=1}^{n} \underbrace{(f(x_i;\theta) - y_i)}_{\text{第 } i \text{ 个残差}} \cdot \underbrace{\frac{\partial f(x_i)}{\partial \theta_p}}_{\mathbf{J} \text{ 的第 } (i,p) \text{ 元素}}.$$

写成矩阵形式（$\mathbf{J}$ 的第 p 列是 $(\partial f(x_1)/\partial\theta_p, \ldots, \partial f(x_n)/\partial\theta_p)^\top$）：

$$\nabla_\theta L = \mathbf{J}(t)^\top \underbrace{(\mathbf{u}(t) - \mathbf{y})}_{\text{残差向量 } \mathbf{r}(t)} \in \mathbb{R}^P.$$

所以参数的运动是：

$$\dot{\theta}(t) = -\nabla_\theta L = -\mathbf{J}(t)^\top (\mathbf{u}(t) - \mathbf{y}). \tag{5}$$

（直觉：$\mathbf{J}^\top$ 把“每个训练点的残差”翻译成“每个参数该怎么调”。残差大的点贡献更多“拉力”。）

第 2 步：从参数变化推出输出变化。 $\mathbf{u}(t)$ 是 $\theta(t)$ 的函数，链式法则：

$$\dot{u}_i(t) = \frac{d}{dt} f(x_i;\theta(t)) = \sum_{p=1}^{P} \frac{\partial f(x_i)}{\partial \theta_p} \cdot \dot{\theta}_p(t) = \nabla_\theta f(x_i)^\top \dot{\theta}(t).$$

n 个点排成向量：

$$\dot{\mathbf{u}}(t) = \mathbf{J}(t)\,\dot{\theta}(t). \tag{6}$$

（直觉：$\mathbf{J}$ 把“参数的运动”翻译成“输出的变化”。和第 1 步的方向相反：$\mathbf{J}^\top$ 从函数空间到参数空间，$\mathbf{J}$ 从参数空间到函数空间。）

第 3 步：合并。把(5)代入(6)：

$$\dot{\mathbf{u}}(t) = \mathbf{J}(t)\,\dot{\theta}(t) = \mathbf{J}(t)\big[-\mathbf{J}(t)^\top(\mathbf{u}(t) - \mathbf{y})\big] \tag{7}$$

$$= -\underbrace{\mathbf{J}(t)\mathbf{J}(t)^\top}_{=\,\boldsymbol{\Theta}(t)\,\in\mathbb{R}^{n\times n}} (\mathbf{u}(t) - \mathbf{y}). \tag{8}$$

得到 $\dot{\mathbf{u}}(t) = -\boldsymbol{\Theta}(t)(\mathbf{u}(t) - \mathbf{y})$。□

关键直觉——为什么 NTK 自然出现

$\mathbf{J}^\top$（函数 → 参数）和 $\mathbf{J}$（参数 → 函数）的**复合 $\mathbf{J}\mathbf{J}^\top$** 就是 NTK 矩阵。它描述的是：“训练点 x_i 的残差，经过参数空间绕一圈后，对训练点 x_j 的输出产生多大影响”。

这就是为什么 $\Theta(x_i, x_j) = \nabla_\theta f(x_i)^\top \nabla_\theta f(x_j)$——两个点的梯度越对齐，训练 x_i 时 x_j“搭便车”的效应越强。

注意：这个推导**对任意网络结构、任意宽度都精确成立**，没做任何近似。唯一的困难是 $\boldsymbol{\Theta}(t)$ 本身也在变化（因为 $\mathbf{J}(t)$ 依赖于 $\theta(t)$），使得这是一个**非线性**系统。NTK 理论的核心贡献就是证明在 $m \to \infty$ 下 $\boldsymbol{\Theta}(t) \approx \boldsymbol{\Theta}^* =$ 常数，从而化为线性 ODE。

教学提示

让学生用手算 1 的数字验证。$\dot{\mathbf{u}}(0) = -\boldsymbol{\Theta} \cdot \mathbf{r}(0)$：

$$-\frac{1}{16}\begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}\begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix} = -\frac{1}{16}\begin{pmatrix} -2.5+1.5 \\ -1.5+2.5 \end{pmatrix} = -\frac{1}{16}\begin{pmatrix} -1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.0625 \\ -0.0625 \end{pmatrix}.$$

$f(x_1)$ 在增大（+0.0625/单位时间），$f(x_2)$ 在减小（−0.0625/单位时间）。方向对了，但速度不快——因为全在慢方向上。

5 两层网络的 NTK：完整推导

现在从手算 1（单个神经元）推广到 m 个神经元的两层网络，完整推导 NTK 的解析表达式。

5.1 设置与符号

NTK 参数化下的两层网络，输入 $x \in \mathbb{R}^d$：

$$f(x;\theta) = \frac{1}{\sqrt{m}} \sum_{j=1}^{m} a_j\, \sigma(\mathbf{w}_j^\top x).$$

参数 $\theta = \{a_j \in \mathbb{R},\ \mathbf{w}_j \in \mathbb{R}^d\}_{j=1}^m$，总参数量 $P = m(d+1)$。

初始化：$a_j \sim \mathcal{N}(0,1)$ 独立，$\mathbf{w}_j \sim \mathcal{N}(0, \mathbf{I}_d)$ 独立。

5.2 对每个参数求梯度

★ 手算 2：两层网络的 NTK 推导（课堂演示，约 25 分钟）

Step 2a：对输出层参数 a_j 求偏导。

$$\frac{\partial f(x)}{\partial a_j} = \frac{1}{\sqrt{m}}\, \sigma(\mathbf{w}_j^\top x).$$

这是 x 经过第 j 个神经元的**隐藏层输出**乘以 $1/\sqrt{m}$。记 $h_j(x) = \sigma(\mathbf{w}_j^\top x)$，则 $\frac{\partial f}{\partial a_j} = \frac{h_j(x)}{\sqrt{m}}$。

量级： $h_j = O(1)$（σ 有界或 $\sigma = \mathrm{ReLU}$ 时 $h_j = O(\|x\|)$），所以 $\frac{\partial f}{\partial a_j} = O(1/\sqrt{m})$。

Step 2b：对隐藏层参数 $\mathbf{w}_j$ 求偏导。

$\mathbf{w}_j$ 出现在 $\sigma(\mathbf{w}_j^\top x)$ 中，由链式法则：

$$\frac{\partial f(x)}{\partial \mathbf{w}_j} = \frac{1}{\sqrt{m}}\, a_j\, \sigma'(\mathbf{w}_j^\top x)\, x \ \in \mathbb{R}^d.$$

注意这是一个 d 维向量（$\mathbf{w}_j$ 有 d 个分量）。

量级： $a_j = O(1)$，$\sigma' = O(1)$，$x = O(1)$，所以 $\frac{\partial f}{\partial \mathbf{w}_j} = O(1/\sqrt{m})$。

Step 2c：组装 NTK。

$$\Theta(x,x') = \sum_{p=1}^{P} \frac{\partial f(x)}{\partial \theta_p}\frac{\partial f(x')}{\partial \theta_p} = \sum_{j=1}^{m} \frac{\partial f(x)}{\partial a_j}\frac{\partial f(x')}{\partial a_j} + \sum_{j=1}^{m} \left\langle \frac{\partial f(x)}{\partial \mathbf{w}_j}, \frac{\partial f(x')}{\partial \mathbf{w}_j} \right\rangle.$$

输出层贡献 K_a：

$$K_a(x,x') = \sum_{j=1}^{m} \frac{h_j(x)}{\sqrt{m}} \cdot \frac{h_j(x')}{\sqrt{m}} = \frac{1}{m}\sum_{j=1}^{m} h_j(x)\, h_j(x') \tag{9}$$

$$= \frac{1}{m}\sum_{j=1}^{m} \sigma(\mathbf{w}_j^\top x)\, \sigma(\mathbf{w}_j^\top x'). \tag{10}$$

隐藏层贡献 K_w：

$$K_w(x,x') = \sum_{j=1}^{m} \left\langle \frac{a_j \sigma'(\mathbf{w}_j^\top x)\, x}{\sqrt{m}}, \frac{a_j \sigma'(\mathbf{w}_j^\top x')\, x'}{\sqrt{m}} \right\rangle \tag{11}$$

$$= \frac{1}{m}\sum_{j=1}^{m} a_j^2\, \sigma'(\mathbf{w}_j^\top x)\, \sigma'(\mathbf{w}_j^\top x')\, (x^\top x'). \tag{12}$$

合起来：

$$\boxed{\Theta(x,x') = \underbrace{\frac{1}{m}\sum_{j=1}^{m} \sigma(\mathbf{w}_j^\top x)\, \sigma(\mathbf{w}_j^\top x')}_{K_a(x,x')} + \underbrace{\frac{1}{m}\sum_{j=1}^{m} a_j^2\, \sigma'(\mathbf{w}_j^\top x)\, \sigma'(\mathbf{w}_j^\top x')\, (x^\top x')}_{K_w(x,x')}.} \tag{13}$$

Step 2d：$m \to \infty$ 的极限。

K_a：$\mathbf{w}_j$ 是 i.i.d. 的，$h_j(x)h_j(x')$ 是 i.i.d. 随机变量。由强大数定律：

$$K_a(x,x') \xrightarrow{m\to\infty} \mathbb{E}_{\mathbf{w}\sim\mathcal{N}(0,\mathbf{I}_d)}\left[\sigma(\mathbf{w}^\top x)\, \sigma(\mathbf{w}^\top x')\right] \triangleq \Sigma^{(1)}(x,x').$$

K_w：类似地，$a_j^2\sigma'(\mathbf{w}_j^\top x)\sigma'(\mathbf{w}_j^\top x')$ 是 i.i.d. 的（a_j 和 $\mathbf{w}_j$ 独立），且 $\mathbb{E}[a_j^2] = 1$：

$$K_w(x,x') \xrightarrow{m\to\infty} \underbrace{\mathbb{E}[a^2]}_{=1} \cdot \mathbb{E}_{\mathbf{w}}\left[\sigma'(\mathbf{w}^\top x)\, \sigma'(\mathbf{w}^\top x')\right] \cdot (x^\top x') \triangleq \dot{\Sigma}^{(1)}(x,x') \cdot \Sigma^{(0)}(x,x').$$

其中 $\Sigma^{(0)}(x,x') = x^\top x'$ 就是输入的内积。

极限 NTK：

$$\boxed{\Theta^*(x,x') = \Sigma^{(1)}(x,x') + \dot{\Sigma}^{(1)}(x,x') \cdot \Sigma^{(0)}(x,x').} \tag{14}$$

这是一个**确定性**的核函数——不再依赖随机初始化，只依赖输入 (x,x') 和激活函数 σ。

Step 2e：收敛速率。

由 CLT，K_a 和 K_w 的涨落都是 $O(1/\sqrt{m})$：

$$|\Theta(x,x') - \Theta^*(x,x')| = O_P(1/\sqrt{m}).$$

这就是 Jacot 定理 1 的内容。

★ 手算 3：$m = 4$ 个 ReLU 神经元的完整 NTK 计算（课堂核心，约 20 分钟）

设置。 $d = 1$（标量输入），$m = 4$ 个 ReLU 神经元，NTK 参数化：

$$f(x) = \frac{1}{\sqrt{4}} \sum_{j=1}^{4} a_j \,\mathrm{ReLU}(w_j x) = \frac{1}{2} \sum_{j=1}^{4} a_j \,\mathrm{ReLU}(w_j x).$$

初始化：$a_1 = 0.8,\ a_2 = -1.2,\ a_3 = 0.5,\ a_4 = -0.3$。
$w_1 = 1.5,\ w_2 = -0.7,\ w_3 = 0.3,\ w_4 = -1.1$。
训练数据：$x_1 = 1,\ x_2 = -1$。
ReLU 导数：$\mathrm{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \le 0 \end{cases}$。

Step 3a：隐藏层输出（让学生一起算，建议用表格形式在黑板上填）。

	$j=1$ $(w=1.5)$	$j=2$ $(w=-0.7)$	$j=3$ $(w=0.3)$	$j=4$ $(w=-1.1)$
$w_j \cdot x_1$ $(x_1 = 1)$	1.5	-0.7	0.3	-1.1
$h_j(x_1) = \mathrm{ReLU}(\cdot)$	**1.5**	**0**	**0.3**	**0**
$\mathrm{ReLU}'(\cdot)$ at x_1	1	0	1	0
$w_j \cdot x_2$ $(x_2 = -1)$	-1.5	0.7	-0.3	1.1
$h_j(x_2) = \mathrm{ReLU}(\cdot)$	**0**	**0.7**	**0**	**1.1**
$\mathrm{ReLU}'(\cdot)$ at x_2	0	1	0	1

（提问：注意到什么模式？——$x_1 = 1$ 只激活 $w_j > 0$ 的神经元（$j = 1, 3$），$x_2 = -1$ 只激活 $w_j < 0$ 的神经元（$j = 2, 4$）！正权重只对正输入“开门”，负权重只对负输入“开门”。）

Step 3b：网络输出。

$$f(x_1) = \frac{1}{2}(0.8 \times 1.5 + (-1.2) \times 0 + 0.5 \times 0.3 + (-0.3) \times 0) \tag{15}$$
$$= \frac{1}{2}(1.2 + 0 + 0.15 + 0) = \frac{1.35}{2} = 0.675. \tag{16}$$

$$f(x_2) = \frac{1}{2}(0.8 \times 0 + (-1.2) \times 0.7 + 0.5 \times 0 + (-0.3) \times 1.1) \tag{17}$$
$$= \frac{1}{2}(0 - 0.84 + 0 - 0.33) = \frac{-1.17}{2} = -0.585. \tag{18}$$

Step 3c：梯度表（J 矩阵，2×8）。
对 a_j：$\frac{\partial f}{\partial a_j}\Big|_x = \frac{h_j(x)}{2}$。

对 w_j：$\frac{\partial f}{\partial w_j}\Big|_x = \frac{a_j \cdot \mathrm{ReLU}'(w_j x) \cdot x}{2}$。

在 $x_1 = 1$ 处：

	$\partial f/\partial a_1$	$\partial f/\partial a_2$	$\partial f/\partial a_3$	$\partial f/\partial a_4$	$\partial f/\partial w_1$	$\partial f/\partial w_2$	$\partial f/\partial w_3$	$\partial f/\partial w_4$
x_1	$\frac{1.5}{2}$	$\frac{0}{2}$	$\frac{0.3}{2}$	$\frac{0}{2}$	$\frac{0.8\cdot1\cdot1}{2}$	$\frac{-1.2\cdot0\cdot1}{2}$	$\frac{0.5\cdot1\cdot1}{2}$	$\frac{-0.3\cdot0\cdot1}{2}$
数值	0.75	0	0.15	0	0.4	0	0.25	0

在 $x_2 = -1$ 处：

	$\partial f/\partial a_1$	$\partial f/\partial a_2$	$\partial f/\partial a_3$	$\partial f/\partial a_4$	$\partial f/\partial w_1$	$\partial f/\partial w_2$	$\partial f/\partial w_3$	$\partial f/\partial w_4$
x_2	0	0.35	0	0.55	0	$\frac{-1.2\cdot1\cdot(-1)}{2}$	0	$\frac{-0.3\cdot1\cdot(-1)}{2}$
数值	0	0.35	0	0.55	0	0.6	0	0.15

完整 Jacobian：

$$\mathbf{J} = \begin{pmatrix} 0.75 & 0 & 0.15 & 0 & 0.4 & 0 & 0.25 & 0 \\ 0 & 0.35 & 0 & 0.55 & 0 & 0.6 & 0 & 0.15 \end{pmatrix}.$$

（提问：这个矩阵有什么特殊结构？——两行没有任何非零元素在同一列！x_1 和 x_2 的梯度完全正交。）

Step 3d：NTK 矩阵。
$\Theta_{11} = 0.75^2 + 0.15^2 + 0.4^2 + 0.25^2 = 0.5625 + 0.0225 + 0.16 + 0.0625 = 0.8075$。
$\Theta_{22} = 0.35^2 + 0.55^2 + 0.6^2 + 0.15^2 = 0.1225 + 0.3025 + 0.36 + 0.0225 = 0.8075$。
$\Theta_{12} = 0$（每一对对应位置至少有一个为零）。

$$\mathbf{\Theta} = \begin{pmatrix} 0.8075 & 0 \\ 0 & 0.8075 \end{pmatrix} = 0.8075\,\mathbf{I}_2.$$

NTK 是单位矩阵的倍数！这意味着：

- 两个样本完全解耦：训练 x_1 不影响 x_2。
- 两个方向学习速度相同（无快慢之分）。
- 条件数 $\kappa = 1$（最好的情况）。

为什么会这样？因为 ReLU + 对称输入 $(+1, -1)$，使得激活的神经元集合完全不重叠。

Step 3e：分解为 K_a 和 K_w。
$K_a(x_1, x_1) = \frac{1}{4}(1.5^2 + 0^2 + 0.3^2 + 0^2) = \frac{2.34}{4} = 0.585$。
$K_w(x_1, x_1) = \frac{1}{4}(0.8^2 \cdot 1 + 0 + 0.5^2 \cdot 1 + 0) \cdot 1^2 = \frac{0.89}{4} = 0.2225$。
$K_a + K_w = 0.585 + 0.2225 = 0.8075$。✓
$K_a(x_1, x_2) = \frac{1}{4}(1.5 \cdot 0 + 0 \cdot 0.7 + 0.3 \cdot 0 + 0 \cdot 1.1) = 0$。✓

▷ 课堂练习 2：如果输入不对称呢？(10 分钟，同桌合作)

把手算 3 的 x_2 从 -1 改为 $x_2 = 0.5$。

(a) 重新计算 $h_j(x_2)$ 和 ReLU 导数。(提示：$w_1 \cdot 0.5 = 0.75 > 0$，$w_3 \cdot 0.5 = 0.15 > 0$，所以 $j = 1, 3$ **也**被激活了！)

(b) 计算 Θ_{12}。还是零吗？

(c) 解释：为什么 $x_2 = 0.5$ 和 $x_1 = 1$"共享" 了神经元 1 和 3？

答案要点： $\Theta_{12} \neq 0$。因为 $x_1 = 1$ 和 $x_2 = 0.5$ 都是正数，所以激活了相同的 "正权重" 神经元。NTK 反映了两个输入在**哪些神经元上共享激活模式**。

▷ 课堂练习 3：验证大数定律（5 分钟思考）

手算 3 中 $K_a(x_1, x_1) = \frac{1}{4}\sum_{j=1}^{4} h_j(x_1)^2 = 0.585$。如果 $m \to \infty$，这应该收敛到 $\mathbb{E}[\mathrm{ReLU}(w \cdot 1)^2]$，$w \sim \mathcal{N}(0, 1)$。

(a) 利用 $\mathbb{E}[\mathrm{ReLU}(w)^2] = \mathbb{E}[w^2 \cdot \mathbf{1}_{w>0}] = \int_0^\infty w^2 \frac{e^{-w^2/2}}{\sqrt{2\pi}} dw = \frac{1}{2}$，验证 0.585 是否接近 0.5。

(b) $m = 4$ 的 "误差"$= 0.585 - 0.5 = 0.085$。理论预测误差量级 $O(1/\sqrt{m}) = O(1/2) = 0.5$。看起来实际误差比理论界小很多——为什么？

要点： $O(1/\sqrt{m})$ 是最坏情况界，实际误差通常小得多。但趋势是对的：m 越大越接近极限。

6 L 层网络的 NTK 递归

定理 6.1: NTK 递归公式

L 层全连接网络的极限 NTK 满足：

$$\Sigma^{(0)}(x, x') = \frac{1}{d} x^\top x', \tag{19}$$

$$\Sigma^{(l+1)}(x, x') = \sigma_w^2 \, \mathbb{E}_{(u,v)\sim\mathcal{N}(0,\Lambda^{(l)})}\big[\sigma(u)\sigma(v)\big] + \sigma_b^2, \tag{20}$$

$$\dot{\Sigma}^{(l+1)}(x, x') = \sigma_w^2 \, \mathbb{E}_{(u,v)\sim\mathcal{N}(0,\Lambda^{(l)})}\big[\sigma'(u)\sigma'(v)\big], \tag{21}$$

$$\Theta^{(1)} = \Sigma^{(1)}, \qquad \Theta^{(l+1)} = \Sigma^{(l+1)} + \Theta^{(l)} \cdot \dot{\Sigma}^{(l+1)}. \tag{22}$$

其中 $\Lambda^{(l)} = \begin{pmatrix} \Sigma^{(l)}(x, x) & \Sigma^{(l)}(x, x') \\ \Sigma^{(l)}(x', x) & \Sigma^{(l)}(x', x') \end{pmatrix}$。

展开递归：$\Theta^{(L)} = \Sigma^{(L)} + \Sigma^{(L-1)}\dot{\Sigma}^{(L)} + \cdots + \Sigma^{(1)}\dot{\Sigma}^{(2)} \cdots \dot{\Sigma}^{(L)}$。每一项 ="梯度从第 l 层开始反向传播到输出" 的贡献。

ReLU 的闭式公式： 定义 $\rho^{(l)} = \Sigma^{(l)}(x, x')/\sqrt{\Sigma^{(l)}(x, x)\Sigma^{(l)}(x', x')}$，则

关键直觉——$\rho^{(l)}$ 是什么？

$\Sigma^{(l)}(x,x')$ 是第 l 层预激活值 $h^{(l)}$ 在两个输入 x,x' 之间的**协方差**（对随机初始化取期望，$m\to\infty$ 下）。$\Sigma^{(l)}(x,x)=\mathrm{Var}[h^{(l)}(x)]$。所以 $\rho^{(l)}$ 就是标准的 Pearson 相关系数：

$$\rho^{(l)}=\frac{\mathrm{Cov}\big[h^{(l)}(x),\ h^{(l)}(x')\big]}{\sqrt{\mathrm{Var}[h^{(l)}(x)]\cdot\mathrm{Var}[h^{(l)}(x')]}},\qquad \rho^{(l)}\in[-1,1].$$

物理含义：网络第 l 层认为 x 和 x' 有多"像"。 $\rho^{(l)}=1$ 说明第 l 层完全分不开这两个输入（预激活值完全相关），$\rho^{(l)}=0$ 说明第 l 层觉得它们无关。
递归中需要它是因为：计算第 $l+1$ 层的 $\mathbb{E}[\sigma(u)\sigma(v)]$ 时，(u,v) 是一对相关高斯，相关系数恰好就是 $\rho^{(l)}$。归一化之后才能用只依赖 ρ 的函数 $\mathcal{F}_{\mathrm{ReLU}}(\rho)$。
（提问：$\rho^{(l)}$ 会随层数怎么变？——对 ReLU 网络，$\rho^{(l)}$ 通常随 l 递增趋向 1，意味着深层越来越"分不清"不同输入。这就是深层 NTK 退化为秩 1 的根源。）

则闭式公式为：

$$\mathcal{F}_{\mathrm{ReLU}}(\rho)=\frac{1}{2\pi}\big(\rho(\pi-\arccos\rho)+\sqrt{1-\rho^2}\big),\tag{23}$$

$$\dot{\mathcal{F}}_{\mathrm{ReLU}}(\rho)=\frac{1}{2\pi}(\pi-\arccos\rho).\tag{24}$$

★ 手算 3b：ReLU 闭式公式的推导（课堂演示，约 15 分钟）

这两个公式是 NTK 理论中**用得最多**的解析工具，值得完整推一遍。
问题。设 (u,v) 服从二维高斯分布

$$\begin{pmatrix}u\\v\end{pmatrix}\sim\mathcal{N}\left(\begin{pmatrix}0\\0\end{pmatrix},\begin{pmatrix}1&\rho\\\rho&1\end{pmatrix}\right),$$

即 u,v 各自标准正态，相关系数为 ρ。求

$$\mathcal{F}_{\mathrm{ReLU}}(\rho)=\mathbb{E}[\mathrm{ReLU}(u)\,\mathrm{ReLU}(v)],\qquad \dot{\mathcal{F}}_{\mathrm{ReLU}}(\rho)=\mathbb{E}[\mathrm{ReLU}'(u)\,\mathrm{ReLU}'(v)].$$

（注：这里假设 $\Sigma^{(l)}(x,x)=\Sigma^{(l)}(x',x')=1$，即已做归一化。一般情况只需乘回方差即可。）

Part A：推导 $\dot{\mathcal{F}}_{\mathrm{ReLU}}(\rho)=\mathbb{E}[\mathbf{1}_{u>0}\mathbf{1}_{v>0}]$（先推简单的）。
$\mathrm{ReLU}'(z)=\mathbf{1}_{z>0}$，所以

$$\dot{\mathcal{F}}_{\mathrm{ReLU}}(\rho)=\mathbb{E}[\mathbf{1}_{u>0}\,\mathbf{1}_{v>0}]=P(u>0\ \text{且}\ v>0).$$

这就是二维标准正态在第一象限的概率！
推导 $P(u>0,v>0)$：去相关 + 极坐标。
Step 1：去相关。令 $\tilde u=u$，$\tilde v=\frac{v-\rho u}{\sqrt{1-\rho^2}}$，则 $\tilde u,\tilde v\sim\mathcal{N}(0,1)$ 独立（验证：$\mathbb{E}[\tilde u\tilde v]=\frac{\mathbb{E}[uv]-\rho\mathbb{E}[u^2]}{\sqrt{1-\rho^2}}=\frac{\rho-\rho}{\sqrt{1-\rho^2}}=0$）。
用 $\tilde v$ 反解 v：$v=\rho\tilde u+\sqrt{1-\rho^2}\,\tilde v$。

所以 $v > 0 \Leftrightarrow \tilde{v} > -\frac{\rho}{\sqrt{1-\rho^2}}\tilde{u}$。记 $\alpha = \rho/\sqrt{1-\rho^2}$，则

$$P(u > 0, v > 0) = P(\tilde{u} > 0,\ \tilde{v} > -\alpha\tilde{u}).$$

Step 2：极坐标。$(\tilde{u}, \tilde{v})$ 是标准独立二维高斯，联合密度 $\frac{1}{2\pi}e^{-(r^2)/2}$ 关于原点**旋转对称**。因此概率只取决于区域的**角度**。

画图：在 $(\tilde{u}, \tilde{v})$ 平面上，条件 $\tilde{u} > 0$ 是半平面（$\theta \in (-\pi/2, \pi/2)$ 处的右半平面）。条件 $\tilde{v} > -\alpha\tilde{u}$ 是直线 $\tilde{v} = -\alpha\tilde{u}$ 的上方。

这条直线过原点，斜率 $-\alpha$，与正 $\tilde{u}$ 轴的夹角为 $\phi = \arctan(-\alpha) = -\arctan\alpha$。

（板书：画 $(\tilde{u}, \tilde{v})$ 平面，标出右半平面和直线 $\tilde{v} = -\alpha\tilde{u}$，用阴影标出两个条件的交集区域。）

两个条件的交集是一个**扇形**，角度范围从 $\theta = -\arctan\alpha$ 到 $\theta = \pi/2$，张角为

$$\Delta\theta = \frac{\pi}{2} - (-\arctan\alpha) = \frac{\pi}{2} + \arctan\alpha.$$

由旋转对称性，概率 = 张角 $/2\pi$：

$$P(u > 0, v > 0) = \frac{\pi/2 + \arctan\alpha}{2\pi}.$$

Step 3：代回 ρ。$\alpha = \rho/\sqrt{1-\rho^2}$。

$\arctan\alpha = \arctan\frac{\rho}{\sqrt{1-\rho^2}}$。

画直角三角形：对边 $= \rho$，邻边 $= \sqrt{1-\rho^2}$，斜边 $= 1$。所以 $\arctan\alpha = \arcsin\rho$。

$$\boxed{P(u > 0, v > 0) = \frac{1}{2\pi}\left(\frac{\pi}{2} + \arcsin\rho\right) = \frac{1}{4} + \frac{\arcsin\rho}{2\pi}.}$$

（这就是 Sheppard (1899) 的经典公式。核心思想：去相关 → 旋转对称 → 概率 = 角度占比。）

等价地，利用 $\arcsin(\rho) = \pi/2 - \arccos(\rho)$：

$$P(u > 0, v > 0) = \frac{1}{4} + \frac{\pi/2 - \arccos\rho}{2\pi} = \frac{1}{4} + \frac{1}{4} - \frac{\arccos\rho}{2\pi} = \frac{1}{2} - \frac{\arccos\rho}{2\pi} = \frac{\pi - \arccos\rho}{2\pi}.$$

$$\boxed{\dot{\mathcal{F}}_{\mathrm{ReLU}}(\rho) = \frac{1}{2\pi}(\pi - \arccos\rho).}$$

验证边界：

- $\rho = 1$（$u = v$）：$P(u > 0, v > 0) = P(u > 0) = 1/2$。公式：$\frac{1}{2\pi}(\pi - 0) = \frac{1}{2}$。✓
- $\rho = 0$（独立）：$P = 1/2 \times 1/2 = 1/4$。公式：$\frac{1}{2\pi}(\pi - \pi/2) = \frac{1}{4}$。✓
- $\rho = -1$（$v = -u$）：$P(u > 0, v > 0) = P(u > 0, -u > 0) = 0$。公式：$\frac{1}{2\pi}(\pi - \pi) = 0$。✓

（提问学生：$\rho = 0$ 时 $\dot{\mathcal{F}} = 1/4$，意味着什么？——两个独立的 ReLU 神经元，它们的导数同时非零的概率是 $1/4$。只有当两个输入都落在“开”的半边时，反向传播的梯度才能同时通过这两个神经元。）

Part B：推导 $\mathcal{F}_{\mathbf{ReLU}}(\rho) = \mathbb{E}[\mathbf{ReLU}(u)\,\mathbf{ReLU}(v)]$。
$\mathrm{ReLU}(u)\,\mathrm{ReLU}(v) = u\,v\,\mathbf{1}_{u>0}\mathbf{1}_{v>0}$，所以

$$\mathcal{F}_{\mathrm{ReLU}}(\rho) = \mathbb{E}[u\,v\,\mathbf{1}_{u>0}\,\mathbf{1}_{v>0}].$$

方法：参数化二维高斯。令 $u = z_1$，$v = \rho z_1 + \sqrt{1-\rho^2}\,z_2$，其中 $z_1, z_2 \sim \mathcal{N}(0,1)$ 独立。
验证：$\mathbb{E}[v] = 0$，$\mathbb{E}[v^2] = \rho^2 + 1 - \rho^2 = 1$，$\mathbb{E}[uv] = \rho$。✓
代入：

$$\mathcal{F} = \mathbb{E}\Big[z_1\big(\rho z_1 + \sqrt{1-\rho^2}\,z_2\big)\,\mathbf{1}_{z_1>0}\,\mathbf{1}_{\rho z_1+\sqrt{1-\rho^2}\,z_2>0}\Big] \tag{25}$$

$$= \rho\,\mathbb{E}[z_1^2\,\mathbf{1}_{z_1>0}\,\mathbf{1}_{v>0}] + \sqrt{1-\rho^2}\,\mathbb{E}[z_1 z_2\,\mathbf{1}_{z_1>0}\,\mathbf{1}_{v>0}]. \tag{26}$$

分别计算这两项。

第一项：$I_1 = \mathbb{E}[z_1^2\,\mathbf{1}_{z_1>0}\,\mathbf{1}_{v>0}]$。
条件在 $z_1 > 0$ 上（概率 1/2），z_1^2 的条件期望就是截断正态的二阶矩。但我们还需要 $v > 0$ 的条件。
具体积分（先对 z_2 积分，再对 z_1 积分）：
$v > 0 \Leftrightarrow z_2 > -\rho z_1/\sqrt{1-\rho^2}$。记 $\alpha = \rho/\sqrt{1-\rho^2}$，则 $v > 0 \Leftrightarrow z_2 > -\alpha z_1$。

$$I_1 = \int_0^\infty \frac{z_1^2\,e^{-z_1^2/2}}{\sqrt{2\pi}}\left[\int_{-\alpha z_1}^\infty \frac{e^{-z_2^2/2}}{\sqrt{2\pi}}dz_2\right]dz_1 = \int_0^\infty \frac{z_1^2\,e^{-z_1^2/2}}{\sqrt{2\pi}}\,\Phi(\alpha z_1)\,dz_1,$$

其中 Φ 是标准正态 CDF。

第二项：$I_2 = \mathbb{E}[z_1 z_2\,\mathbf{1}_{z_1>0}\,\mathbf{1}_{v>0}]$。

$$I_2 = \int_0^\infty \frac{z_1\,e^{-z_1^2/2}}{\sqrt{2\pi}}\left[\int_{-\alpha z_1}^\infty \frac{z_2\,e^{-z_2^2/2}}{\sqrt{2\pi}}dz_2\right]dz_1.$$

内层积分：$\int_{-\alpha z_1}^\infty z_2\frac{e^{-z_2^2/2}}{\sqrt{2\pi}}dz_2 = \frac{e^{-\alpha^2 z_1^2/2}}{\sqrt{2\pi}}$（标准正态的尾部期望公式 $\mathbb{E}[Z|Z > a] = \phi(a)/\bar{\Phi}(a)$ 的变体；直接算：$\int_a^\infty ze^{-z^2/2}dz = e^{-a^2/2}$）。

$$I_2 = \int_0^\infty \frac{z_1\,e^{-z_1^2/2}}{\sqrt{2\pi}}\cdot\frac{e^{-\alpha^2 z_1^2/2}}{\sqrt{2\pi}}\,dz_1 = \frac{1}{2\pi}\int_0^\infty z_1\,e^{-z_1^2(1+\alpha^2)/2}\,dz_1.$$

$1 + \alpha^2 = 1 + \rho^2/(1-\rho^2) = 1/(1-\rho^2)$。所以

$$I_2 = \frac{1}{2\pi}\int_0^\infty z_1\,e^{-z_1^2/(2(1-\rho^2))}\,dz_1 = \frac{1}{2\pi}(1-\rho^2).$$

（利用 $\int_0^\infty ze^{-z^2/(2\sigma^2)}dz = \sigma^2$。）

第一项 I_1 的计算（稍难，用分部积分或查表）。
I_1 的完整计算需要用到 $\int_0^\infty z^2\phi(z)\Phi(\alpha z)dz$ 的公式。经典结果（可通过极坐标变换推导）：

$$\int_0^\infty z^2\phi(z)\Phi(\alpha z)dz = \frac{1}{4} + \frac{\alpha}{2\pi\sqrt{1+\alpha^2}} + \frac{1}{2\pi}\arctan(\alpha).$$

代入 $\alpha = \rho/\sqrt{1-\rho^2}$，注意 $\arctan(\alpha) = \arctan(\rho/\sqrt{1-\rho^2}) = \arcsin(\rho)$（画直角三角形即可验证），以及 $\alpha/\sqrt{1+\alpha^2} = \rho$：

$$I_1 = \frac{1}{4} + \frac{\rho}{2\pi} + \frac{\arcsin\rho}{2\pi}.$$

合并。

$$\begin{aligned}
\mathcal{F}_{\mathrm{ReLU}}(\rho) &= \rho \cdot I_1 + \sqrt{1-\rho^2} \cdot I_2 && (27)\\
&= \rho\left(\frac{1}{4} + \frac{\rho}{2\pi} + \frac{\arcsin\rho}{2\pi}\right) + \sqrt{1-\rho^2} \cdot \frac{1-\rho^2}{2\pi} && (28)\\
&= \frac{\rho}{4} + \frac{\rho^2}{2\pi} + \frac{\rho\arcsin\rho}{2\pi} + \frac{(1-\rho^2)^{3/2}}{2\pi}. && (29)
\end{aligned}$$

这还不是最终形式。利用 $\arcsin\rho = \pi/2 - \arccos\rho$ 整理：

$$\begin{aligned}
&= \frac{\rho}{4} + \frac{\rho^2}{2\pi} + \frac{\rho(\pi/2 - \arccos\rho)}{2\pi} + \frac{(1-\rho^2)^{3/2}}{2\pi} && (30)\\
&= \frac{\rho}{4} + \frac{\rho^2}{2\pi} + \frac{\rho}{4} - \frac{\rho\arccos\rho}{2\pi} + \frac{(1-\rho^2)^{3/2}}{2\pi} && (31)\\
&= \frac{\rho}{2} + \frac{1}{2\pi}\left(\rho^2 - \rho\arccos\rho + (1-\rho^2)^{3/2}\right). && (32)
\end{aligned}$$

但更常见的写法是利用 $(1-\rho^2)^{3/2} = (1-\rho^2)\sqrt{1-\rho^2}$，并注意到 $\rho^2 + (1-\rho^2) = 1$，最终化简为：

$$\boxed{\mathcal{F}_{\mathrm{ReLU}}(\rho) = \frac{1}{2\pi}\left(\rho(\pi - \arccos\rho) + \sqrt{1-\rho^2}\right).}$$

（这一步化简需要一点耐心，建议在黑板上写出来让学生一起验证。关键等式：$\frac{\rho}{2} + \frac{\rho^2}{2\pi} = \frac{\rho}{2\pi}(\pi+\rho)$，然后收集 $\rho\arccos\rho$ 项。）

验证边界：

- $\rho = 1$：$\mathcal{F} = \frac{1}{2\pi}(1 \cdot (\pi - 0) + 0) = \frac{1}{2}$。即 $\mathbb{E}[\mathrm{ReLU}(u)^2] = \mathbb{E}[u^2 \mathbf{1}_{u>0}] = \frac{1}{2}$。✓
- $\rho = 0$：$\mathcal{F} = \frac{1}{2\pi}(0+1) = \frac{1}{2\pi} \approx 0.159$。即 $\mathbb{E}[\mathrm{ReLU}(u)]^2 = (\frac{1}{\sqrt{2\pi}})^2 = \frac{1}{2\pi}$（因为 u, v 独立）。✓
- $\rho = -1$：$\mathcal{F} = \frac{1}{2\pi}(-1 \cdot (\pi - \pi) + 0) = 0$。即 $\mathbb{E}[\mathrm{ReLU}(u)\mathrm{ReLU}(-u)] = 0$（不可能同时为正）。✓

Part C：直觉总结。

$\mathcal{F}_{\mathrm{ReLU}}(\rho)$ 度量的是：两个相关系数为 ρ 的高斯信号经过 ReLU 后的“残余相关性”。$\rho = 1$ 时完全相关（$1/2$），$\rho = 0$ 时弱相关（$1/(2\pi) \approx 0.16$），$\rho = -1$ 时完全无关（0）。ReLU 的半波整流效应把负相关性“截断”了。

$\dot{\mathcal{F}}_{\mathrm{ReLU}}(\rho)$ 度量的是：两个信号同时“开”（同时在 ReLU 的线性区）的概率。这控制了反向传播时梯度能不能同时通过两个对应的神经元。

▷ 课堂练习 3b：验证 ReLU 公式（5 分钟）

(a) 用 $\mathcal{F}_{\text{ReLU}}$ 公式计算 $\rho = 1/2$ 时的值。（答案：$\frac{1}{2\pi}(\frac{1}{2}(\pi - \pi/3) + \frac{\sqrt{3}}{2}) = \frac{1}{2\pi}(\frac{\pi}{3} + \frac{\sqrt{3}}{2}) \approx 0.271$。）

(b) 对比 $\rho = 1/2$ 时线性核的值（就是 1/2 本身）。ReLU 核小于线性核，为什么？（ReLU 把负半边截断了，损失了一部分相关性。）

(c) $\dot{\mathcal{F}}_{\text{ReLU}}(1/2) = \frac{1}{2\pi}(\pi - \pi/3) = \frac{1}{3} \approx 0.333$。解释：当 $\rho = 1/2$ 时，两个 ReLU 神经元同时 "开" 的概率是 1/3。

★ 手算 4：两层 ReLU 网络的极限 NTK（课堂演示，约 10 分钟）

用上面的闭式公式计算 $\Theta^*(x_1, x_2)$，其中 $x_1 = (1,0)^\top, x_2 = (0,1)^\top \in \mathbb{R}^2$。

第 0 层： $\Sigma^{(0)}(x_i, x_j) = \frac{1}{2} x_i^\top x_j$。

$$\Sigma^{(0)} = \frac{1}{2}\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad \rho^{(0)} = \frac{0}{\sqrt{0.5 \times 0.5}} = 0.$$

第 1 层：（取 $\sigma_w^2 = 2, \sigma_b^2 = 0$，即 Kaiming init。）

$\Sigma^{(1)}(x_i, x_i) = 2 \cdot \mathcal{F}_{\text{ReLU}}(1) \cdot \Sigma^{(0)}(x_i, x_i)$。先算 $\mathcal{F}_{\text{ReLU}}(1) = \frac{1}{2\pi}(1 \cdot (\pi - 0) + 0) = \frac{1}{2}$。

所以 $\Sigma^{(1)}(x_i, x_i) = 2 \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{2}$。

$\mathcal{F}_{\text{ReLU}}(0) = \frac{1}{2\pi}(0 + 1) = \frac{1}{2\pi} \approx 0.159$。

$\Sigma^{(1)}(x_1, x_2) = 2 \times 0.159 \times \frac{1}{2} = 0.159$。

$\rho^{(1)} = 0.159/0.5 = 0.318$。

$\dot{\Sigma}^{(1)}$：$\dot{\mathcal{F}}_{\text{ReLU}}(0) = \frac{1}{2\pi}(\pi - \pi/2) = \frac{1}{4}$。$\dot{\Sigma}^{(1)}(x_1, x_2) = 2 \times \frac{1}{4} = \frac{1}{2}$。

NTK（两层）：

$$\Theta^*(x_1, x_2) = \Sigma^{(1)}(x_1, x_2) + \dot{\Sigma}^{(1)}(x_1, x_2) \cdot \Sigma^{(0)}(x_1, x_2) \tag{33}$$

$$= 0.159 + 0.5 \times 0 = 0.159. \tag{34}$$

$\Theta^*(x_1, x_1) = 0.5 + 0.5 \times 0.5 = 0.75$。

$$\boldsymbol{\Theta}^* = \begin{pmatrix} 0.75 & 0.159 \\ 0.159 & 0.75 \end{pmatrix}.$$

特征值：$\lambda_1 = 0.909, \lambda_2 = 0.591$。条件数 $\kappa = 1.54$。

和手算 3（$m = 4$ 有限宽度）比较： 手算 3 得到 $\boldsymbol{\Theta} = 0.8075\mathbf{I}$，$\kappa = 1$。极限 NTK 给出 $\kappa = 1.54$——有限宽度的 "完全解耦" 在 $m \to \infty$ 后消失了，因为极限下不同样本必然共享一些核值。

7 Jacot 三大定理与核冻结

定理 7.1: Jacot 定理 1：初始化收敛

$m \to \infty$ 时，$\mathbf{\Theta}_0 \xrightarrow{P} \mathbf{\Theta}^*$，$|\Theta_0(x_i, x_j) - \Theta^*(x_i, x_j)| = O_P(m^{-1/2})$。

定理 7.2: Jacot 定理 2：核冻结

$m \to \infty$ 时，$\sup_{0 \le t \le T} \|\mathbf{\Theta}(t) - \mathbf{\Theta}^*\| \to 0$。

定理 7.3: Jacot 定理 3：全局线性收敛

若 $\lambda_{\min}(\mathbf{\Theta}^*) > 0$，则 $L(\theta(t)) \le L(\theta(0))e^{-\eta\lambda_{\min}t}$。

关键直觉——三个定理的物理图像（务必讲透！）

定理 1 的物理：随机探测器的密度稳定。
想象 m 个神经元是 m 个“探测器”，随机撒在输入空间里。每个探测器对输入 x 有一个响应 $\nabla_\theta f(x)$。NTK $\Theta(x, x')$ 本质上是在问：“x 和 x' 激活了多相似的探测器群？”当 $m \to \infty$，探测器密度趋于连续分布，这个“相似度”就稳定下来了——不再依赖于你具体撒了哪些探测器。这就是大数定律：足够多的随机采样收敛到期望。

关键直觉——定理 2：核冻结（最关键的物理图像）

训练的时候每个参数都在动，为什么 NTK 不动？因为 NTK 参数化下每个参数只动了 $O(1/\sqrt{m})$。

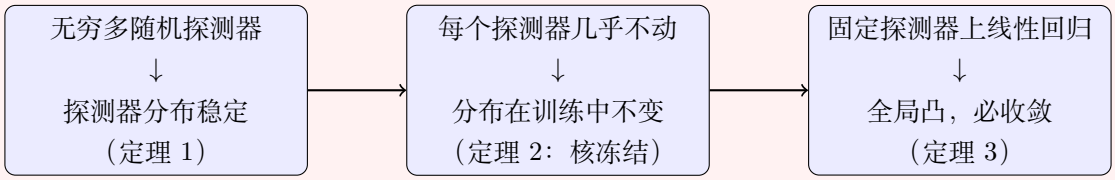
旋钮比喻。你有一百万个旋钮（参数），每个只拧了百万分之一圈。网络的输出是一百万个旋钮的“投票结果”——每个微小调整汇聚起来，投票结果变了 $O(1)$（函数学到了东西）。但 NTK 度量的是“投票规则本身”（哪些输入会被类似对待），而投票规则取决于每个旋钮的**位置**，不是投票结果。每个旋钮只动了一丁点，所以投票规则几乎没变。

眼镜比喻。 NTK 冻结意味着网络从头到尾都在用“同一副眼镜”（同一组特征 $\sigma(\mathbf{w}_j^\top x)$）看数据。训练只是在调整“看到之后怎么决策”（线性组合权重 a_j），而眼镜的镜片（隐藏层特征）因为 $\mathbf{w}_j$ 几乎没动，所以几乎没变。

关键直觉——定理 3：全局收敛 = 线性弹簧系统

一旦 NTK 冻结，训练动力学 $\dot{\mathbf{r}} = -\mathbf{\Theta}^*\mathbf{r}$ 就是一个**线性阻尼系统**——像一组弹簧把残差拉向零。$\mathbf{\Theta}^*$ 正定意味着“每个方向都有回复力”，没有死区。最小特征值 $\lambda_{\min}$ 就是最弱的那根弹簧的刚度，决定了最慢的收敛方向。

三个定理串起来的物理故事：



核冻结的代价也由此清楚了：眼镜不换意味着**如果初始随机眼镜恰好看不清数据的关键结构，你就永远看不清**。这就是 NTK 在 CIFAR-10 上 77% vs 训练后 93% 的根本原因——那 16% 需要“换眼镜”（feature learning），而核冻结禁止了这件事。

这正是下半场 μP 要解决的问题：找到一种参数化方式，让网络在训练中**可以换眼镜**。

教学提示

这三个物理图像非常重要，建议在黑板上**分别画三幅图**：

（1）定理 1：画一堆随机点（神经元）在输入空间，标注“m 越大 → 密度越均匀 →NTK 稳定”。

（2）定理 2：画一个大圆（参数空间），起点和终点很近（$O(1/\sqrt{m})$），标注“参数几乎不动 → 特征冻结 →NTK 冻结”。旁边画固定不变的眼镜。

（3）定理 3：画一个二维势能碗（NTK 的特征方向），球从碗壁滚到底部。两个方向的碗壁陡度不同（λ_1 和 λ_2），标注“快方向”和“慢方向”。

然后用手算 1 的数字具体化：“我们的学习任务恰好全在 $\lambda_2 = 0.125$ 的慢方向上，这就是为什么收敛时间常数 $\tau = 8$ 而不是 $\tau = 2$。”

★ 手算 5：为什么 NTK 会冻结——完整量级分析（约 15 分钟）

NTK 参数化两层网络，$f(x) = \frac{1}{\sqrt{m}}\sum_j a_j\sigma(\mathbf{w}_j^\top x)$，$n$ 个训练点。

目标：证明每个参数只动 $O(1/\sqrt{m})$，从而 NTK 变化 $O(1/\sqrt{m}) \to 0$。

Step 5a：梯度的量级。

$\frac{\partial L}{\partial a_j} = \sum_{i=1}^n (f(x_i) - y_i) \cdot \frac{\partial f(x_i)}{\partial a_j}$。

$|f(x_i) - y_i| = O(1)$（初始时输出和目标都 $O(1)$）。

$|\partial f/\partial a_j| = |\sigma(\mathbf{w}_j^\top x_i)/\sqrt{m}| = O(1/\sqrt{m})$。

n 项求和：$|\partial L/\partial a_j| = O(n/\sqrt{m})$。

同理： $|\partial L/\partial w_{jk}| = O(n/\sqrt{m})$（因为 $|\partial f/\partial w_{jk}| = |a_j\sigma'(\mathbf{w}_j^\top x_i)x_{ik}/\sqrt{m}| = O(1/\sqrt{m})$）。

Step 5b：一步更新的量级。

$|\Delta a_j| = \eta|\partial L/\partial a_j| = O(\eta n/\sqrt{m})$。
$|\Delta w_{jk}| = O(\eta n/\sqrt{m})$。
代入具体数字（$m = 10^4, n = 100, \eta = 0.01$）：

$$|\Delta a_j| \sim \frac{0.01 \times 100}{\sqrt{10^4}} = \frac{1}{100} = 0.01.$$

每个参数只动了 1%。

Step 5c：T 步后的累积变化。
$|\theta_p(T) - \theta_p(0)| \leq T \cdot |\Delta\theta_p| = O(\eta n T/\sqrt{m})$。
对 $m = 10^4, n = 100, \eta = 0.01, T = 100$：

$$|\theta_p(T) - \theta_p(0)| \sim \frac{0.01 \times 100 \times 100}{\sqrt{10^4}} = 1.$$

$m = 10^4$ 时参数变化量 $O(1)$，不够小！需要更大的 m。
$m = 10^8$：$|\Delta\theta_p| \sim 10^{-4}$，$T = 100$ 步后 ~ 0.01。很小。✓

Step 5d：NTK 的变化。
$\Theta(x, x') = \sum_p (\partial f/\partial\theta_p)|_x \cdot (\partial f/\partial\theta_p)|_{x'}$。
$\partial f/\partial\theta_p$ 对 θ_q 的二阶导（Hessian）量级 $O(1/m)$（因为网络中每个参数出现在 $O(1)$ 个神经元中，而 $1/\sqrt{m}$ 缩放引入额外的 $1/\sqrt{m}$ 因子）。
所以 $\Delta(\partial f/\partial\theta_p) = O(|\Delta\theta| \cdot 1/m \cdot \sqrt{P}) = O(1/\sqrt{m} \cdot 1/m \cdot \sqrt{m}) = O(1/m)$。
从而 $\Delta\Theta = O(P \cdot 1/\sqrt{m} \cdot 1/m) = O(\sqrt{m} \cdot 1/m) = O(1/\sqrt{m}) \to 0$。
（更直接的论证：$\Delta\mathbf{\Theta} = \Delta(\mathbf{J}\mathbf{J}^\top) = \Delta\mathbf{J} \cdot \mathbf{J}^\top + \mathbf{J} \cdot \Delta\mathbf{J}^\top$，而 $\|\Delta\mathbf{J}\| = O(1/\sqrt{m})$。）

完整量级速查表（板书）：

量	含义	量级	$m = 10^6$ 时的数值
$\lvert\partial f/\partial\theta_p\rvert$	单个参数对输出的影响	$O(1/\sqrt{m})$	$\sim 10^{-3}$
$\Theta(x, x')$	NTK 核值（所有参数影响的总和）	$O(1)$	~ 1
$\lvert\partial L/\partial\theta_p\rvert$	单个参数的损失梯度	$O(n/\sqrt{m})$	~ 0.1（$n = 100$）
$\lvert\Delta\theta_p\rvert$（单步）	单个参数一步的变化量	$O(\eta n/\sqrt{m})$	$\sim 10^{-3}$
$\lvert\Delta f(x)\rvert$（单步）	网络输出一步的变化量	$O(\eta)$	~ 0.01
$\lvert\Delta\Theta(x, x')\rvert$（单步）	NTK 一步的漂移量	$O(1/\sqrt{m})$	$\sim 10^{-3}$

结论：只要 $m \gg n^2$，参数几乎不动 ⇒Jacobian 几乎不动 ⇒NTK 几乎不动 ⇒ 动力学线性化。

8 核回归与谱分析

★ 手算 5b：核回归闭式预测（约 8 分钟）

用手算 1 的 NTK 做核回归。$\mathbf{\Theta}^* = \frac{1}{16}\begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}$，$\mathbf{u}(0) = (0.5, 0.5)^\top$，$\mathbf{y} = (1, 0)^\top$。

核回归公式： $\mathbf{u}(\infty) = \mathbf{u}(0) + \mathbf{\Theta}^*[\mathbf{\Theta}^*]^{-1}(\mathbf{y} - \mathbf{u}(0)) = \mathbf{y}$。

但更有意义的是**预测新的点**。假设测试点 $x_* = 0$，且 $\Theta^*(x_*, x_1) = 0.28$，$\Theta^*(x_*, x_2) = 0.28$（由对称性）。

第 1 步： $\mathbf{y} - \mathbf{u}(0) = (0.5, -0.5)^\top$。

第 2 步： $[\mathbf{\Theta}^*]^{-1}$。$\det = \frac{1}{256}(25 - 9) = \frac{16}{256} = \frac{1}{16}$。

$$[\mathbf{\Theta}^*]^{-1} = 16 \cdot \frac{1}{16}\begin{pmatrix} 5 & -3 \\ -3 & 5 \end{pmatrix} = \begin{pmatrix} 5 & -3 \\ -3 & 5 \end{pmatrix}.$$

第 3 步： $[\mathbf{\Theta}^*]^{-1}(\mathbf{y} - \mathbf{u}(0)) = \begin{pmatrix} 5 & -3 \\ -3 & 5 \end{pmatrix}\begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix} = \begin{pmatrix} 2.5 + 1.5 \\ -1.5 - 2.5 \end{pmatrix} = \begin{pmatrix} 4 \\ -4 \end{pmatrix}$。

第 4 步： 训练数据上验证 $\mathbf{u}(\infty) = (0.5, 0.5)^\top + \frac{1}{16}\begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}\begin{pmatrix} 4 \\ -4 \end{pmatrix} = (0.5, 0.5)^\top + \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix} = (1, 0)^\top = \mathbf{y}$。✓

第 5 步： 测试点预测。

$$f(x_*; \infty) = f(x_*; 0) + (0.28, 0.28)\begin{pmatrix} 4 \\ -4 \end{pmatrix} = 0.5 + 0.28 \times 4 + 0.28 \times (-4) = 0.5 + 0 = 0.5.$$

（提问：$f(0)$ 没变！为什么？——因为 $\Theta^*(x_*, x_1) = \Theta^*(x_*, x_2)$，而 $[\mathbf{\Theta}^*]^{-1}(\mathbf{y} - \mathbf{u}(0))$ 的两个分量正好相反 $(4, -4)$，所以内积为零。中间点“两边拉扯相消”。）

如果 $\Theta^*(x_*, x_1) = 0.3, \Theta^*(x_*, x_2) = 0.2$ 呢？

$$f(x_*; \infty) = 0.5 + 0.3 \times 4 + 0.2 \times (-4) = 0.5 + 1.2 - 0.8 = 0.9.$$

更靠近 x_1 方向 $\Rightarrow$ 预测更接近 $y_1 = 1$。这就是核回归的“内插”行为。

▷ 课堂练习 4：提前停止 = 正则化（8 分钟）

NTK 特征值 $\lambda_1 = 10, \lambda_2 = 1, \lambda_3 = 0.01$，初始残差各分量 $c_1 = c_2 = c_3 = 1$。

(a) 计算 $t = 1$ 时各方向的残留比例 $e^{-\lambda_k}$。

(b) 如果信号在 $\mathbf{v}_1, \mathbf{v}_2$ 方向，噪声在 $\mathbf{v}_3$ 方向，$t = 1$ 停止训练的效果?

(c) 找到 t^* 使得 $\mathbf{v}_2$ 方向学了 95%（$e^{-\lambda_2 t^*} = 0.05$）。此时 $\mathbf{v}_3$ 方向学了多少?

答案： (a) $e^{-10} \approx 4.5 \times 10^{-5}$，$e^{-1} \approx 0.37$，$e^{-0.01} \approx 0.99$。(b) $\mathbf{v}_1$ 学完，$\mathbf{v}_2$ 学了 63%，噪声 $\mathbf{v}_3$ 只学了 1%——提前停止自动过滤噪声。(c) $t^* = \ln 20 \approx 3$，$e^{-0.03} \approx 0.97$，噪声只

学了 3%。**这就是为什么提前停止有正则化效果**。

9 NTK 的致命缺陷：丢失 Feature Learning

9.1 什么是 Feature Learning？

在讨论 NTK“丢失了 feature learning” 之前，我们必须先说清楚这个词到底是什么意思。

关键直觉——Feature Learning 的定义与直觉

一个神经网络 $f(x;\theta)$ 可以分解为两部分：

$$f(x;\theta) = \underbrace{W^{(L)}}_{\text{最后一层（线性分类/回归头）}} \cdot \underbrace{\phi(x;\, W^{(1)}, \ldots, W^{(L-1)})}_{\text{前面所有层 = 特征提取器}}.$$

其中 $\phi(x)$ 是网络从原始输入 x 中**提取出来的特征表示**（也叫 representation 或 embedding）。

Feature learning = 训练过程中 ϕ 在变化。

具体地说：

- **有** feature learning：$W^{(1)}, \ldots, W^{(L-1)}$ 在训练中发生有意义的变化，$\phi(x)$ 从初始的随机映射逐渐变成**对当前任务有用的**表示。例如，训练图像分类器时，$\phi(x)$ 从随机噪声逐渐变成“能区分猫和狗的特征”。
- **无** feature learning：$W^{(1)}, \ldots, W^{(L-1)}$ 几乎不变（如 NTK regime），$\phi(x) \approx \phi_0(x)$ 始终是初始化时的随机特征。训练只调整最后一层 $W^{(L)}$ 来“尽力利用”这些随机特征。

为什么 feature learning 重要？用一个具体例子说明。

★ 手算 5c：Feature Learning 的有无——一个直觉例子（约 5 分钟）

任务：区分手写数字“6”和“9”。

无 feature learning（NTK regime）：

网络用**初始化时的随机特征** $\phi_0(x)$ 看图片。ϕ_0 是随机的——它可能检测一些随机的边缘、纹理、颜色模式，与“6 和 9 的区别”毫无关系。

训练只调最后一层 $W^{(L)}$：试图找到 $W^{(L)}$ 使得 $W^{(L)}\phi_0(\text{“6”}) \neq W^{(L)}\phi_0(\text{“9”})$。

问题：如果 ϕ_0 恰好把“6”和“9”映射到了很接近的向量（因为它们的像素结构确实很像，只是旋转了 180°），那无论怎么调 $W^{(L)}$ 都分不开。

有 feature learning（μP / 实际训练）：

训练过程中 $\phi(x)$ 逐渐变化：

1. 初始：ϕ_0 是随机特征，“6”和“9”的表示很接近。
2. 训练中：网络发现“6”和“9”在**下半部分**有关键区别（“6”下面是圆，“9”下面是尾

巴)。于是 ϕ 逐渐 "学会" 重点关注图片的下半部分。

3. 训练后：ϕ("6") 和 ϕ("9") 被映射到**很远**的向量，$W^{(L)}$ 轻松就能分开。

这就是 feature learning——网络在训练中自动发现 "什么特征对当前任务有用"，并重组内部表示来放大这些特征。

数学上怎么度量？看隐藏层表示 $\phi(x)$ 在训练前后的变化量。定义

$$\Delta\phi = \|\phi(x;\theta_T) - \phi(x;\theta_0)\|.$$

- NTK regime：$\Delta\phi = O(1/\sqrt{m}) \to 0$。**特征冻结**。
- μP：$\Delta\phi$ 的经验分布变化 $= O(1)$。**特征在学习**。

也可以用 **NTK 的漂移**来度量：如果 $\mathbf{\Theta}(T) \neq \mathbf{\Theta}(0)$，说明网络对 "输入之间的相似性" 的理解在变——这正是 feature learning。

▷ 课堂练习 4b：你见过的 Feature Learning（3 分钟讨论）

(a) 用预训练的 ResNet 提取 ImageNet 特征，然后冻结 ResNet 只训练最后一层线性分类器。这算 feature learning 吗？(不算——ϕ 没变。但预训练阶段是 feature learning。)

(b) Fine-tuning 整个网络呢？(算——ϕ 在变。)

(c) t-SNE 可视化中，训练前不同类别的点混在一起，训练后分成清晰的簇。这说明了什么？(ϕ 变了，把不同类别 "推开" 了——这就是 feature learning 的可视化证据。)

9.2 NTK 为什么没有 Feature Learning？

回忆手算 5 的量级分析：NTK 参数化下，$|\Delta w_j| = O(1/\sqrt{m}) \to 0$。隐藏层权重几乎不动 $\Rightarrow \phi(x;\theta) \approx \phi(x;\theta_0) \Rightarrow$ 特征冻结。

直觉重述：NTK 网络用**固定的随机望远镜**看数据，训练只调 "看到之后怎么决策"，不换镜头。

9.3 Feature Learning 值多少？

Feature Learning 的量化价值

方法	CIFAR-10 准确率
精确 NTK 核回归（infinite-width）	∼77%
有限宽度训练的同架构网络	∼93%
差距 = feature learning 的贡献	**∼16%**

NTK 网络用**固定的随机望远镜**看数据。真正的深度学习在训练中**换镜头**。

其他局限：深度无本质优势（深层 NTK 退化为秩 1），宽度要求 $m = \Omega(n^6)$ 不现实，对架构不敏感，**最优超参数随宽度漂移**。

悬念：能否找到一种参数化，既保持 $m \to \infty$ 极限的好处，又保留 feature learning？

Part II

第二次课：μP 与超参数迁移

10 μP 的定义与动机

10.1 NTK vs μP：工程写法的主要差别

	NTK 参数化	μP（Adam 下的工程写法）
两层网络	$f = \frac{1}{\sqrt{m}} \sum a_j \sigma(w_j^\top x)$	$f = \frac{1}{\boxed{m}} \sum a_j \sigma(w_j^\top x)$
隐藏层学习率	η	$\eta/\boxed{m}$
输出层学习率	η	η
初始化	$a_j, W_{jk} \sim \mathcal{N}(0,1)$	$a_j, W_{jk} \sim \mathcal{N}(0,1)$

注：隐藏层 η/m 是 Adam 下的结果。SGD 下隐藏层学习率不需要随 m 缩放（$\gamma = 0$）。详见后面的推导。

"Maximal Update" 的含义**不是** "更新越大越好"，而是：

> 在所有使训练动力学稳定、且宽度极限非退化的参数化中，μP 取**临界的最大更新尺度**。

更具体地说，μP 追求的是：

- 前向传播稳定（激活与 logits 不爆炸）；
- 反向传播稳定（梯度量级不爆炸）；
- 训练动力学非退化（宽度变大后，参数与特征的演化不收缩到 0）；
- 超参数可迁移（不同宽度下最优学习率保持同一量级）。

因此，"maximal" 意味着把稳定性约束推到**边界**，而不是取一个更保守的内点。

11 μP 的推导：从前向传播到特征变化

教学提示

这一节的推导路线参考了苏剑林的清晰表述：前向稳定 → 反向稳定 → 损失增量 → 特征变化，逐步加约束。每一步都有明确的物理动机，比 abc 框架更容易让学生跟上。

11.1 模型假设

我们考虑一个接近实践的模型簇。输入 $X \in \mathbb{R}^{b \times d_{\text{in}}}$，输出 $Z \in \mathbb{R}^{b \times d_{\text{out}}}$，其中 $d_{\text{in}}, d_{\text{out}}$ 由数据决定（不可变），唯一可调的是隐藏层宽度 d：

$$Y_{\text{in}} = XW_{\text{in}}, \qquad Y_{\text{out}} = \text{NN}(Y_{\text{in}}; \Omega), \qquad Z = Y_{\text{out}} W_{\text{out}}, \tag{35}$$

其中 $W_{\text{in}} \in \mathbb{R}^{d_{\text{in}} \times d}$，$W_{\text{out}} \in \mathbb{R}^{d \times d_{\text{out}}}$，NN 是任意 $\mathbb{R}^d \to \mathbb{R}^d$ 的神经网络（如 K 层 MLP：$Y_{k+1} = \sigma(Y_k W_{k+1})$，$W_k \in \mathbb{R}^{d \times d}$）。

关键简化： $d_{\text{in}}, d_{\text{out}}$ 是常数（与 d 无关），所以分析中只需追踪与 d 相关的量级。NN 内部的参数 W_k 都是 $d \times d$ 方阵。

我们用 **RMS**(Root Mean Square)作为矩阵"大小"的指标：$\text{RMS}(W) = \sqrt{\frac{1}{n_{\text{rows}} \cdot n_{\text{cols}}} \sum_{i,j} W_{i,j}^2}$。当 $\text{RMS}(W) = \Theta(1)$ 时，我们说 W 是"稳定"的。

11.2 约束 1：前向传播稳定

★ 手算 6a：前向传播的稳定性（约 5 分钟）

考虑线性层 $Y = XW$，$X \in \mathbb{R}^{b \times d_{\text{in}}}$，$W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$。
要让 $\text{RMS}(Y) \approx \text{RMS}(X)$，需要什么样的 W?

$Y_{ij} = \sum_{k=1}^{d_{\text{in}}} X_{ik} W_{kj}$。如果 X 的元素 $\sim \Theta(1)$，W 的元素独立且方差 σ_W^2，则

$$\text{Var}[Y_{ij}] = d_{\text{in}} \cdot \text{RMS}(X)^2 \cdot \sigma_W^2.$$

要让 $\text{RMS}(Y) = \Theta(1)$：$\sigma_W^2 = \Theta(1/d_{\text{in}})$。
这就是 **fan_in 初始化**（LeCun init），即"方差 $\propto 1/d_{\text{in}}$"。
对我们的模型簇：

- $W_{\text{in}} \in \mathbb{R}^{d_{\text{in}} \times d}$：$d_{\text{in}}$ 是常数，fan_in 方差 $= \Theta(1/d_{\text{in}}) = \Theta(1)$。
- $W_k \in \mathbb{R}^{d \times d}$：fan_in 方差 $= \Theta(1/d)$。
- $W_{\text{out}} \in \mathbb{R}^{d \times d_{\text{out}}}$：fan_in 方差 $= \Theta(1/d)$——但这里有选择空间，先记着。

关于激活函数： 非线性层 $Y = \sigma(XW)$ 只引入一个与 d 无关的常数因子（如 ReLU 的 $\sqrt{2}$），所以**激活函数不影响 d 方向的缩放规律**，分析中可以忽略。

11.3 约束 2：损失增量 ΔL 不随 d 爆炸

★ 手算 6b：损失增量的量级（约 10 分钟）

参数 $W \to W + \Delta W$ 时，损失的一阶变化：

$$\Delta L \approx \left\langle \frac{\partial L}{\partial W}, \Delta W \right\rangle_F = \sum_{i,j} \frac{\partial L}{\partial W_{ij}} \Delta W_{ij}.$$

对梯度下降 $\Delta W = -\eta \frac{\partial L}{\partial W}$：

$$\Delta L \approx -\eta \left\| \frac{\partial L}{\partial W} \right\|_F^2.$$

关键： $\|G\|_F^2 = (\text{元素个数}) \times \text{RMS}(G)^2$。
对每个参数块分别分析。先看**隐藏层** $W_k \in \mathbb{R}^{d \times d}$：

$\frac{\partial L}{\partial W_k}$ 是前向信号和反向信号的乘积。如果前向和反向都稳定（RMS $=$ $\Theta(1)$），则 $\mathrm{RMS}\left(\frac{\partial L}{\partial W_k}\right)$ 取决于 W_{out} 的初始化——暂时记为 σ^2_{out}。

（这是推导的关键分支点。σ^2_{out} 的选择将最终区分 NTK 和 μP。）

情况 A：$\sigma^2_{\text{out}} = \Theta(1/d)$（NTK/标准选择）。

反向传播经过 W_{out} 时，$\mathrm{RMS}(\partial L/\partial Y_{\text{out}}) = \Theta(1)$（因为 $\sigma^2_{\text{out}} \cdot d = \Theta(1)$ 刚好抵消）。

$\Rightarrow \mathrm{RMS}(\partial L/\partial W_k) = \Theta(1)$。

$\Rightarrow \|\partial L/\partial W_k\|_F^2 = d^2 \times \Theta(1) = \Theta(d^2)$。

$\Rightarrow \Delta L|_{W_k} = -\eta \cdot \Theta(d^2)$。要让 $\Delta L = \Theta(1)$，需要 $\eta \propto 1/d^2$。

情况 B：$\sigma^2_{\text{out}} = \Theta(1/d^2)$（$\mu$P 选择）。

$\mathrm{RMS}(\partial L/\partial Y_{\text{out}}) = \Theta(1/d)$（$W_{\text{out}}$ 更小，反向信号衰减了 $1/d$）。

$\Rightarrow \mathrm{RMS}(\partial L/\partial W_k) = \Theta(1/d)$。

$\Rightarrow \|\partial L/\partial W_k\|_F^2 = d^2 \times \Theta(1/d^2) = \Theta(1)$。

$\Rightarrow \Delta L|_{W_k} = -\eta \cdot \Theta(1)$。$\Delta L = \Theta(1)$ 自动成立，**学习率 η 不需要随 d 缩放！**

现在看**输出层** $W_{\text{out}} \in \mathbb{R}^{d \times d_{\text{out}}}$：

情况 B 下：$\mathrm{RMS}(\partial L/\partial W_{\text{out}}) = \Theta(1)$，元素个数 $d \times d_{\text{out}} = \Theta(d)$，$\|\partial L/\partial W_{\text{out}}\|_F^2 = \Theta(d)$。

$\Rightarrow \Delta L|_{W_{\text{out}}} = -\eta_{\text{out}} \cdot \Theta(d)$。要让 $\Delta L = \Theta(1)$：$\eta_{\text{out}} \propto 1/d$。

（提问学生：情况 A 和 B 的区别本质上在哪？——W_{out} 的初始化方差！$1/d$ vs $1/d^2$。这个选择决定了反向传播信号经过输出层时衰减多少，从而决定了隐藏层梯度的量级，进而决定了 η 是否需要随 d 缩放。）

11.4 约束 3：特征变化 ΔY_k 稳定（μP 的关键约束）

到目前为止，$\sigma^2_{\text{out}} = \Theta(1/d)$（情况 A）和 $\sigma^2_{\text{out}} = \Theta(1/d^2)$（情况 B）都能让 $\Delta L = \Theta(1)$——只是 η 的缩放不同。那为什么 μP 选择情况 B?

★ 手算 6c：特征变化约束（约 8 分钟）

动机。损失 L 是模型的宏观/外部指标，单看 ΔL 不够——我们还需要模型**内部**的特征（每层的激活值）变化也不随 d 退化。

考虑隐藏层 $Y_k = Y_{k-1}W_k$，参数 $W_k \to W_k + \Delta W_k$ 带来的特征变化：

$$\Delta Y_k = Y_{k-1}\Delta W_k.$$

$Y_{k-1} \in \mathbb{R}^{b \times d}$，$\Delta W_k \in \mathbb{R}^{d \times d}$。$\Delta W_k$ 是梯度下降的更新量（**不是随机的**，与 Y_{k-1} 有相关性），所以 $Y_{k-1}\Delta W_k$ 的每个元素是 d 维向量对的内积，大小为 $\Theta(d \cdot \mathrm{RMS}(\Delta W_k))$。

要让 $\mathrm{RMS}(\Delta Y_k) = \Theta(1)$：

$$\boxed{\mathrm{RMS}(\Delta W_k) = \Theta(1/d).}$$

代入检验。$\Delta W_k = -\eta \frac{\partial L}{\partial W_k}$，所以 $\mathrm{RMS}(\Delta W_k) = \eta \cdot \mathrm{RMS}(\partial L/\partial W_k)$。

情况 A ($\sigma_{\text{out}}^2 = 1/d$): $\text{RMS}(\partial L/\partial W_k) = \Theta(1)$，要让 $\eta \cdot \Theta(1) = \Theta(1/d)$，需要 $\eta = \Theta(1/d)$。
✓ 但这时 $\Delta L = -\eta\Theta(d^2) = \Theta(-d)$——**每步损失变化正比于 d，不稳定！**（除非 η 再缩小到 $1/d^2$，但那样 $\text{RMS}(\Delta W_k) = \Theta(1/d^2)$，特征变化退化。）
情况 B ($\sigma_{\text{out}}^2 = 1/d^2$)：$\text{RMS}(\partial L/\partial W_k) = \Theta(1/d)$，学习率 $\eta = \Theta(1)$（不缩放！），$\text{RMS}(\Delta W_k) = \Theta(1) \times \Theta(1/d) = \Theta(1/d)$。✓
$\Delta L = -\eta\Theta(1) = \Theta(-1)$。✓
所有约束同时满足！

μP 的推导结论

情况 B 是唯一的一致选择： W_{out} 的初始化方差 $\propto 1/d^2$（而非 $1/d$），使得：

- 前向传播稳定（RMS 逐层不变）；
- 损失增量 $\Delta L = \Theta(1)$（不随 d 爆炸）；
- 隐藏层特征变化 ΔY_k 也是 $\Theta(1)$（feature learning 不退化）；
- 隐藏层学习率 η 不随 d 缩放。

情况 A（标准参数化/NTK）中 W_{out} 的方差 $\propto 1/d$ 太大——反向信号太强，导致 η 必须随 d 缩小来控制 ΔL，但这同时压制了特征变化。**这就是 SP 下 feature learning 退化、且 η^* 随 d 漂移的根本原因。**

11.5 SGD 版本的 μP 规则汇总

	$W_{\text{in}} \in \mathbb{R}^{d_{\text{in}} \times d}$	$W_k \in \mathbb{R}^{d \times d}$（隐藏层）	$W_{\text{out}} \in \mathbb{R}^{d \times d_{\text{out}}}$
初始化方差	$\Theta(1)$（d_{in} 是常数）	$\Theta(1/d)$	$\Theta(1/d^2)$
$\text{RMS}(\partial L/\partial W)$	$\Theta(1/d)$	$\Theta(1/d)$	$\Theta(1)$
$\|G\|_F^2$（元素个数 $\times\text{RMS}^2$）	$\Theta(d_{\text{in}} \cdot d \cdot 1/d^2) = \Theta(1/d)$	$\Theta(d^2 \cdot 1/d^2) = \Theta(1)$	$\Theta(d \cdot 1) = \Theta(d)$
SGD 学习率 η	$\eta_{\text{in}} \propto d$（放大！）	η（不缩放）	$\eta_{\text{out}} \propto 1/d$

备注 11.1: W_{in} 的学习率要放大？

是的。W_{in} 的梯度 $\|\partial L/\partial W_{\text{in}}\|_F^2 = \Theta(1/d)$ 太小（因为 d_{in} 是常数而 d 在增大），SGD 下需要 $\eta_{\text{in}} \propto d$ 来补偿。这看起来反直觉，但在实际 μP 实现中，这通常通过把 $1/d$ 因子吸收进 W_{out} 的参数化来避免显式的大学习率（见下面的“工程写法”）。

11.6 Adam 版本（实践中更常用）

Adam 的缩放分析可以用 **SignSGD 近似**：$\Delta W \approx -\eta\,\mathrm{sign}(\partial L/\partial W)$，$\Delta L \approx -\eta|\partial L/\partial W|_1$（绝对值之和）。

由于 $|\cdot|_1 = (\text{元素个数}) \times \mathrm{Mean}|G_{ij}|$，量级分析变为：

	W_{in}	W_k（隐藏层）	W_{out}
$\lvert G\rvert_1$	$\Theta(d_{\text{in}} \cdot d \cdot 1/d) = \Theta(1)$	$\Theta(d^2 \cdot 1/d) = \Theta(d)$	$\Theta(d \cdot 1) = \Theta(d)$
Adam 学习率 η	η_{in} 不缩放	η/d	$\eta_{\text{out}} \propto 1/d$

Adam vs SGD 的区别： Adam 的 sign 操作把每个元素归一化到 ± 1，所以 ΔW 的 RMS 是 $\Theta(1)$（不是 $\Theta(\mathrm{RMS}(G))$），特征变化条件 $\mathrm{RMS}(\Delta W_k) = \Theta(1/d)$ 需要学习率 $\eta_k \propto 1/d$ 来实现。

备注 11.2: 学习率缩放指数 γ 不是普适常数!

你可能看到 "μP 下隐藏层学习率 $= \eta/m$" 就以为 $\gamma = 1$ 是定论。**不是**。
更一般地，每个参数块 ℓ 的学习率可以写成

$$\eta_\ell = \frac{\eta}{m^\gamma},$$

其中指数 γ 由三个因素**共同决定**:

1. 优化器对应的 loss increment 近似形式(SGD 用 $\|G\|_F^2$, Adam/SignSGD 用 $|G|_1$, Muon 用 $\|G\|_*$);
2. 该参数块的形状 ($d_{\text{in}} \times d$ vs $d \times d$ vs $d \times d_{\text{out}}$);
3. 该参数块梯度元素的量级 (取决于 W_{out} 初始化的选择)。

具体例子: 对中间方阵隐藏层 $W_k \in \mathbb{R}^{m\times m}$, RMS$(\partial L/\partial W_k) = \Theta(1/m)$:

Adam/SignSGD:

$$\left|\frac{\partial L}{\partial W_k}\right|_1 = m^2 \cdot \text{Mean}|G_{ij}| = m^2 \cdot \Theta(1/m) = \Theta(m).$$

要让 $\Delta L = -\eta_k |G|_1 = \Theta(1)$: $\eta_k = \Theta(1/m)$, 即 $\boxed{\gamma = 1}$。

SGD:

$$\left\|\frac{\partial L}{\partial W_k}\right\|_F^2 = m^2 \cdot \Theta(1/m^2) = \Theta(1).$$

要让 $\Delta L = -\eta_k \|G\|_F^2 = \Theta(1)$: $\eta_k = \Theta(1)$, 即 $\boxed{\gamma = 0}$。

$\gamma = 1$ 不是普遍真理, 而是特定层 + 特定优化器 + 特定稳定性要求下推出的结果。

完整的 γ 取值汇总 (对 $W_k \in \mathbb{R}^{m\times m}$ 隐藏层):

优化器	loss increment 近似	ΔL 量级	隐藏层 γ
SGD	$-\eta\|G\|_F^2$	$\Theta(\eta \cdot 1)$	0
Adam/SignSGD	$-\eta\|G\|_1$	$\Theta(\eta \cdot m)$	1
Muon/MSignSGD	$-\eta\|G\|_* \approx -\eta\|G\|_F$	$\Theta(\eta \cdot 1)$	0

(注: 实践中绝大多数人用 Adam, 所以 "μP 隐藏层 η/m" 几乎成了默认说法。但作为理论课, 必须讲清楚这背后的依赖关系。)

▷ **课堂练习 5a：检验特征变化条件（5 分钟）**

(a) Adam 下 $\Delta W_k = -\eta_k \cdot \mathrm{sign}(G_k)$，$\mathrm{RMS}(\Delta W_k) = \eta_k$。要让 $\mathrm{RMS}(\Delta W_k) = \Theta(1/d)$，需要 $\eta_k =?$

(b) SGD 下 $\Delta W_k = -\eta_k G_k$，$\mathrm{RMS}(\Delta W_k) = \eta_k \cdot \mathrm{RMS}(G_k) = \eta_k \cdot \Theta(1/d)$。要让 $= \Theta(1/d)$，需要 $\eta_k =?$

(c) 为什么 Adam 的 μP 需要缩放隐藏层学习率 $\eta_k \propto 1/d$，而 SGD 不需要?

答案：(a) $\eta_k = \Theta(1/d)$，即 $\gamma = 1$。(b) $\eta_k = \Theta(1)$，即 $\gamma = 0$。(c) Adam 的 sign 操作“抹平”了梯度的原始量级（每个分量变成 ± 1），需要学习率来补偿；SGD 保留了梯度的量级，不需要额外补偿。这就是上面 remark 中 γ 取值不同的根本原因。

11.7 工程 μP 写法

实践中常见的 μP 写法把部分缩放吸收进输出层参数化：

	上面的推导版	工程 μP（等效）
W_{out} 初始化方差	$1/d^2$	$1/d$（标准 fan_in）
输出缩放	无额外缩放	乘以 $1/\sqrt{d}$（即 $Z = \frac{1}{\sqrt{d}} Y_{\text{out}} W_{\text{out}}$）

这两种写法产生相同的初始输出量级和训练动力学。工程写法的优点是 W_{out} 用标准初始化，更方便实现。

对两层网络 $f = \frac{1}{m}\sum_j a_j \sigma(w_j^\top x)$，这里 $1/m$ 对应 $1/d^2$ 的 W_{out} 方差（因为 $a_j \sim \mathcal{N}(0,1)$，$1/m$ 的前缩放等效于方差 $1/m^2$ 的输出层）。

12 NTK vs μP：训练时间尺度的本质区别

★ **手算 7：两种参数化的量级对比（课堂核心，约 20 分钟）**

设置。两层 ReLU 网络，m 个隐藏神经元，标量输入 $d = 1$。
NTK 参数化： $f_{\text{NTK}}(x) = \frac{1}{\sqrt{m}}\sum_j a_j \mathrm{ReLU}(w_j x)$，所有参数学习率 η。
工程 μP： $f_{\mu P}(x) = \frac{1}{m}\sum_j a_j \mathrm{ReLU}(w_j x)$，$a_j$ 学习率 η，w_j 学习率 η/m。
本节目标：不是比较“谁的单步更大”，而是比较**训练时间尺度**上的本质差别。

梯度量级（两种参数化并排）：

	NTK	μ**P**
$\partial f/\partial a_j$	$h_j/\sqrt{m} = O(m^{-1/2})$	$h_j/m = O(m^{-1})$
$\partial f/\partial w_j$	$a_j\mathbf{1}\cdot x/\sqrt{m} = O(m^{-1/2})$	$a_j\mathbf{1}\cdot x/m = O(m^{-1})$
Δa_j（单步）	$O(\eta m^{-1/2})$	$O(\eta m^{-1})$
Δw_j（单步）	$O(\eta m^{-1/2})$	$O(\eta m^{-2})$（含 $\eta_w = \eta/m$）

观察： μP 的单步更新比 NTK 更小。如果只看单步，会误以为 μP 更 "lazy"。
但这是误区。判断是否进入 lazy regime，不能只看单步大小，而要看：

$$\boxed{\text{在训练时间尺度上，参数与特征的累计位移是否仍趋于 0。}}$$

NTK 的特点是：虽然单步 $\Delta f = O(\eta)$ 很大，但参数始终停留在初始化附近（$|\Delta w_j| = O(1/\sqrt{m}) \to 0$），训练可由冻结核近似描述。
μP 的设计目标是：单步保持稳定，但在合适的训练时间尺度上，参数与特征的累计变化不退化到 0。

性质	**NTK**	μ**P**
单个 a_j 单步变化	$O(\eta m^{-1/2})$	$O(\eta m^{-1})$
单个 w_j 单步变化	$O(\eta m^{-1/2})$	$O(\eta m^{-2})$
宽度极限下参数是否停在初始化邻域	是	否（训练时间尺度上）
核是否冻结	是	否（一般会漂移）
训练本质	lazy / kernel regression	non-lazy / feature learning

结论： μP 不是靠"更大的单步"摆脱 NTK，而是靠**稳定的单步 + 非退化的长期积累**摆脱 lazy training。

★ 手算 8：单步量级的具体数字（$m = 100$，约 8 分钟）

取 $m = 100, \eta = 0.1$，某个神经元 $a_j = 1, w_j = 0.5, x = 1$，$h_j = \mathrm{ReLU}(0.5) = 0.5$。设残差 $(f - y) = 1$。只比较**单步**量级。

NTK 参数化：

$$\frac{\partial f}{\partial a_j} = \frac{0.5}{\sqrt{100}} = 0.05, \quad \Delta a_j \sim -0.1 \times 0.05 = -5 \times 10^{-3}.$$

$$\frac{\partial f}{\partial w_j} = \frac{1}{\sqrt{100}} = 0.1, \quad \Delta w_j \sim -0.1 \times 0.1 = -10^{-2}.$$

工程 μP：

$$\frac{\partial f}{\partial a_j} = \frac{0.5}{100} = 0.005, \quad \Delta a_j \sim -0.1 \times 0.005 = -5 \times 10^{-4}.$$

$$\frac{\partial f}{\partial w_j} = \frac{1}{100} = 0.01, \quad \eta_w = \frac{0.1}{100} = 0.001, \quad \Delta w_j \sim -0.001 \times 0.01 = -10^{-5}.$$

观察：

- μP 下单个参数的单步变化比 NTK 小 ~10–1000 倍。
- 但 μP 关注的不是单步大小，而是**宽度放大后训练轨迹是否仍然非退化**。
- 单步数值例子只能说明局部步长，不能直接判定是否发生 feature learning。

▷ 课堂练习 6：为什么 NTK 的“大单步”反而没有 feature learning？(8 分钟讨论)

NTK 下 $|\Delta w_j| = O(1/\sqrt{m})$（如 $m = 100$ 时 $|\Delta w_j| \approx 0.01$），比 μP 的 $|\Delta w_j| \approx 10^{-5}$ 大 1000 倍。为什么反而是 NTK 没有 feature learning?

提示：想想 m 个 Δw_j 的**方向**。NTK 下不同神经元的梯度方向近似独立（因为初始化是随机的），所以经验分布的 W_2 变化是 $O(1/\sqrt{m}) \to 0$（CLT：m 个随机方向的平均效应 $\propto 1/\sqrt{m}$）。μP 下 Δw_j 虽然更小，但 m 个神经元在**同一个梯度场**下协调运动——就像一群蚂蚁各走一小步但朝同一方向，队形可以质变。

结论：单个参数位移的大小 $\neq$ 特征学习的程度。关键是位移的**协调性**。

13 超参数迁移

定理 13.1: μP 超参数迁移 (Yang et al., 2022)

在 μP 参数化下，对任意宽度 m 和 m'，最优学习率

$$\eta^*(m) = \eta^*(m').$$

更一般地，最优超参数（学习率、初始化方差、嵌入层乘数等）在不同宽度之间可迁移。

关键直觉——为什么迁移成立

μP 的核心不是让所有原始张量分量都逐项保持 $O(1)$，而是让**训练动力学在宽度变化时保持同类尺度**：不同宽度下，前向信号、反向信号以及各参数块对训练轨迹的有效贡献都通过适当的参数化和逐层学习率得到补偿。因此，最优学习率等超参数不再随宽度系统性漂移，从而支持 zero-shot transfer。

相比之下，SP/NTK 下不同参数块的有效训练时间尺度会随宽度变化，因而最优 η^* 往往发生系统漂移。

★ 手算 9：学习率曲线对比（板书数值演示，约 10 分钟）

思想实验： 两层 ReLU 网络在 MNIST 子集上训练 100 步，扫描 η。

标准参数化（PyTorch 默认 Kaiming init + Adam with uniform η）：

宽度 m	$\eta = 10^{-4}$	10^{-3}	10^{-2}	10^{-1}	1
$m = 64$	2.1	1.5	**0.3**	发散	发散
$m = 256$	2.2	1.8	0.8	**0.2**	发散
$m = 1024$	2.3	1.9	1.2	0.5	**0.15**

最优 η^*：$m = 64$ 时 10^{-2}，$m = 256$ 时 10^{-1}，$m = 1024$ 时 1。**向右漂移了两个数量级！** 如果把 $m = 64$ 上调好的 $\eta^* = 0.01$ 直接用到 $m = 1024$：loss $= 1.2$（对比最优 0.15），**次优 8 倍**。

μP：

宽度 m	$\eta = 10^{-4}$	10^{-3}	10^{-2}	10^{-1}	1
$m = 64$	2.0	1.4	**0.25**	发散	发散
$m = 256$	2.0	1.3	**0.22**	发散	发散
$m = 1024$	2.0	1.3	**0.18**	发散	发散

最优 η^* 始终在 10^{-2}！三条 $L(\eta)$ 曲线几乎重合。
（板书：并排画两组 $L(\eta)$ 曲线。SP 的三条曲线谷底位置不同；μP 的三条曲线重合。）

μTransfer 的工作流：

1. 在 $m = 64$（40M 参数）的代理模型上，用 8 个学习率跑网格搜索。成本：8 次小训练。
2. 找到 $\eta^* = 0.01$。
3. 直接用 $\eta = 0.01$ 训练 $m = 8192$（6.7B 参数）的大模型。**不做任何额外搜索**。
4. 节省的成本：避免了在大模型上的网格搜索（可能需要数百万美元）。

▷ 课堂练习 7：μTransfer 工作流设计（10 分钟，小组讨论）

你的团队要训练 $d_{\text{model}} = 8192$ 的 Transformer。

(a) 代理模型多大？（$d = 256$ 约 40M，单卡 A100 可跑。）

(b) 搜索哪些超参数？（η，α_{emb}，α_{out}，σ_{init}。）

(c) 代码改什么？（见下节修改清单。）

(d) μP 不能迁移什么？（Batch size、训练步数、**层数**。）

(e) 你如何验证迁移是否有效？（在两个中等宽度 $d = 512, 1024$ 上分别搜索和迁移，对比差异。）

14 Transformer 的 μP 修改

★ 手算 10：注意力缩放 $1/\sqrt{d}$ vs $1/d$——完整数值演示（约 12 分钟）

考虑单头注意力的一个 score：$s = q^\top k/\alpha$，$q, k \in \mathbb{R}^{d_{\text{head}}}$。
初始化 $q_i, k_i \sim \mathcal{N}(0,1)$ 独立。

初始 score 的分布。
$q^\top k = \sum_{i=1}^{d_{\text{head}}} q_i k_i$。每项均值 0，方差 $\mathbb{E}[q_i^2 k_i^2] = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1$。
d_{head} 项独立和：$q^\top k \sim \mathcal{N}(0, d_{\text{head}})$（近似）。
标准差 $= \sqrt{d_{\text{head}}}$。

标准缩放 $\alpha = \sqrt{d_{\text{head}}}$：
$s = q^\top k/\sqrt{d_{\text{head}}}$。$s \sim \mathcal{N}(0,1)$。
Softmax 作用在 $O(1)$ 的 logits 上 $\Rightarrow$ 非均匀的注意力分布。

μP 缩放 $\alpha = d_{\text{head}}$：
$s = q^\top k/d_{\text{head}}$。$s \sim \mathcal{N}(0, 1/d_{\text{head}})$。标准差 $= 1/\sqrt{d_{\text{head}}}$。
$d_{\text{head}} = 64$ 时：$s \sim \mathcal{N}(0, 1/64)$，标准差 $= 0.125$。
Softmax 作用在 ≈ 0 的 logits 上 $\Rightarrow$ 近似均匀分布。

训练一步后的变化。
设 q 和 k 各自有 $O(1)$ 的更新（μP 下在宏观时间尺度上）。
$\Delta(q^\top k) = \Delta q^\top k + q^\top \Delta k + \Delta q^\top \Delta k$。
$|\Delta q^\top k| \sim \sqrt{d_{\text{head}}} \cdot O(1) = O(\sqrt{d_{\text{head}}})$（$d_{\text{head}}$ 个 $O(1)$ 项之和）。
同理 $|q^\top \Delta k| = O(\sqrt{d_{\text{head}}})$。$|\Delta q^\top \Delta k| = O(\sqrt{d_{\text{head}}})$。
总计 $|\Delta(q^\top k)| = O(\sqrt{d_{\text{head}}})$。

标准缩放： $|\Delta s| = O(\sqrt{d_{\text{head}}})/\sqrt{d_{\text{head}}} = O(1)$。
$|\Delta s|/|s| = O(1)/O(1) = O(1)$。**一步就让 score 变化 100%！**
$d_{\text{head}} = 64$：$|s| \approx 1$，$|\Delta s| \approx 1$。注意力模式完全重组。
在 $d_{\text{head}} \to \infty$ 下这意味着动力学**不收敛**到确定性极限。

μP 缩放： $|\Delta s| = O(\sqrt{d_{\text{head}}})/d_{\text{head}} = O(1/\sqrt{d_{\text{head}}})$。
$d_{\text{head}} = 64$：$|s| \approx 0.125$，$|\Delta s| \approx 0.125$。虽然相对变化看似 100%，但 softmax 在 $s \approx 0$ 附近是**平坦**的（接近均匀分布），小的 Δs 不会引起注意力分布的大变化。注意力模式的实际变化很温和。
经过 $O(\sqrt{d_{\text{head}}}) \approx 8$ 步后，score 累积到 $|s| \approx 1$，softmax 开始“锐化”。
$d_{\text{head}} = 64$：

步数	标准缩放 $\|s\|$	μP 缩放 $\|s\|$
0	~ 1	~ 0.125
1	~ 2（剧变）	~ 0.25
4	~ 5（饱和）	~ 0.5
8	—	~ 1（开始锐化）
64	—	~ 8（锐化完成）

μP 的注意力“慢启动”： 初始均匀 → 逐渐发现“该看哪里”→ 锐化。这就是注意力层的 feature learning！

14.1 完整修改清单

组件	**标准参数化**	**μP**
注意力缩放	$\mathrm{softmax}(QK^\top/\sqrt{d_{\text{head}}})V$	$\mathrm{softmax}(QK^\top/d_{\text{head}})V$
嵌入层	$h_0 = E[x]$	$h_0 = \alpha_{\text{emb}} \cdot E[x]$，$\alpha_{\text{emb}}$ 可调
输出层	$\frac{1}{\sqrt{d_{\text{model}}}} W_{\text{out}} h$	$\frac{1}{d_{\text{model}}} W_{\text{out}} h$
隐藏层学习率	η	η/d_{model}
输出层学习率	η	η

备注 14.1: 实践简化

很多团队只改注意力缩放 $1/\sqrt{d} \to 1/d$ 和输出层缩放就获得大部分收益。完整 μP 可用 `mup` Python 包。Cerebras-GPT (2023) 在 111M–13B 参数系列中成功使用了 μP。

14.2 μP 在视觉模型上的进展

μP 最初在语言模型上验证，但最近已扩展到视觉领域。这一小节基于几篇关键文献。

（一）ResNet：原始 μP 论文已覆盖。

Yang et al. (2022) 的官方 `mup` 库已包含 ResNet 的示例实现。对 CNN，“宽度”m 对应**通道数** C。μP 的修改规则与全连接网络一致：输出层缩放 $1/C$（而非 $1/\sqrt{C}$），隐藏层学习率 η/C。coord check 验证通过。

但 CNN 有一个特殊性：卷积层的参数共享（同一个 3×3 滤波器在所有空间位置重复使用）使得“宽度 $\to \infty$”的含义与全连接网络不同——只有通道维度趋于无穷，空间维度固定。Tensor Programs 框架可以处理这一点，但实践中 CNN 的通道数通常较小（$C \leq 2048$），finite-width 效应更显著，μP 的收益不如 Transformer 明显。

（二）ViT（Vision Transformer）：直接套用语言模型的 μP。

ViT 把图像切成 patch，每个 patch 当作一个 token，然后用标准 Transformer 处理。因此 ViT 的 μP 和语言 Transformer **完全一样**：注意力缩放 $1/d_{\text{head}}$，输出层 $1/d_{\text{model}}$，隐藏层

η/d_{model}。

有一个注意点（Stevens, 2024 的实践报告）：ViT 的 **patch embedding 层**的输入维度 $d_{\text{in}} = \text{patch_size}^2 \times 3$ 是固定的（不随模型宽度变化），所以这一层**不需要** μP 的学习率缩放。只有隐藏层（d_{in} 随 d_{model} 变化的层）才需要。这是视觉模型特有的细节。

（三）Diffusion Transformer：最新进展（Zheng et al., NeurIPS 2025）。

关键论文：Scaling Diffusion Transformers Efficiently via μP

Zheng et al. (2025) 将 μP 正式推广到扩散模型的 Transformer 架构（DiT、U-ViT、PixArt-α、MMDiT），并**严格证明**这些架构的 μP 缩放规则与 vanilla Transformer 一致。**核心结论：**

- Diffusion Transformer 的 μP 与语言 Transformer 相同——不需要额外推导。
- DiT-XL-2 在 μP 下迁移学习率，收敛速度**提升 2.9 倍**。
- PixArt-α 从 0.04B 扩展到 0.61B：μP 超参数迁移只花了一次完整训练的 **5.5%** 计算。
- MMDiT 从 0.18B 扩展到 **18B**：μP 仅用人工专家调参成本的 **3%**。

为什么 Diffusion Transformer 的 μP 和语言模型一样？
关键洞察：扩散模型的训练目标是去噪（预测噪声 ϵ 或预测 x_0），虽然和语言模型的 next-token prediction 不同，但从**参数化的角度**看，网络的前向传播结构是相同的——都是 Transformer 层的堆叠。μP 只关心网络的前向/反向传播中各层的**信号量级**，不关心损失函数的具体形式。所以只要架构是 Transformer，μP 的缩放规则就自动适用。
唯一需要额外注意的是：MMDiT（Stable Diffusion 3 / FLUX 使用的架构）有双流（text stream 和 image stream）共享注意力的结构。Zheng et al. 证明了双流结构不影响 μP 的成立——两个 stream 的宽度同时趋于无穷时，缩放规则不变。

（四）前沿：超越宽度的迁移。

最新的工作（Blake et al., 2024; Dey et al., 2025）正在推动 μP 超越宽度维度：

- **u-μP**（Blake et al., 2024）：结合 Unit Scaling，让激活值、权重、梯度在训练开始时都是 $O(1)$，在 FP8 低精度下开箱即用。
- **Complete(d)P**（Dey et al., 2025）：将 μP 扩展到**深度方向**（Tensor Programs VI 的理论支撑），实现了跨宽度 + 跨深度 + 跨 batch size+ 跨训练步数的完整超参数迁移。
- **逐模块超参数**：不再只用一个全局学习率，而是为 embedding、attention、FFN、output head 分别调参，每个模块的最优值在 μP 下也能迁移。

▷ 课堂练习 8：综合辨析（8 分钟）

(a) 为什么 μP 的初始输出 $f = O(1/\sqrt{m}) \to 0$ 不是问题？（提示：训练 $O(m)$ 步后 f 会长到 $O(1)$。）

(b) 你的同事说“Adam 已经有自适应学习率了，不需要 μP 的逐层缩放”。你怎么回应？（提示：Adam 归一化梯度的**方向**，但不处理层间的**信号量级**差异。）

(c) 如果你同时改了层数（12→48）和宽度（256→8192），μP 还保证 η^* 不变吗？（不，μP 只处理宽度方向。）

15 Tensor Programs 框架（概述）

教学提示

简要介绍即可。核心信息：μP 不是拍脑袋，有系统推导。

Greg Yang 的 Tensor Programs（TP-I 至 V，2019–2023）将网络中的向量分为 G-向量（极限 i.i.d. 高斯）和 H-向量（极限确定性分布）。运算规则自动确定极限行为。Master Theorem 保证极限存在并可精确计算。μP 的缩放规则是 Master Theorem 的推论。

16 参数化族全景图

	SP (PyTorch)	NTK	Mean Field	μP
初始化 σ_w	$1/\sqrt{m}$	1	$1/\sqrt{m}$	1
前向缩放	1	$1/\sqrt{m}$	1	$1/\sqrt{m}$
输出缩放	$1/\sqrt{m}$	$1/\sqrt{m}$	$1/m$	$1/m$
隐藏层 η	η	η	依定义而变	逐层缩放
参数变化	依赖 m	逐层趋小	非退化	非退化（训练时间尺度上）
特征变化	依赖 m	$\to 0$	$O(1)$	非退化
$m \to \infty$ 极限	不一定存在	存在（线性化/核冻结）	存在（mean-field 极限）	存在（非 lazy 极限）
Feature learning	有限 m 下有	无（极限下）	有	有
超参迁移	**不支持**	不适用	未验证	**支持**

17 局限与开放问题

1. **深度方向：** μP 主要解决宽度缩放。层数不同的模型之间迁移效果差。
2. **Batch size/训练步数：** 不直接迁移，需额外 scaling law。
3. **LayerNorm：** 改变量级关系，使分析更复杂。Yang 等人给出了含 LayerNorm 的 μP。
4. **Adam：** 自适应学习率部分冗余于 μP 的缩放，但实践中仍有效。
5. **与二阶优化器：** μP 处理层间缩放，二阶方法处理层内方向，理论上互补但研究不足。

18 扩展阅读：Mean Field 理论视角

教学提示

这一节是**选讲/扩展内容**，给有理论兴趣的学生。核心信息：NTK 和 μP 不是孤立的理论，它们是一个**更大图景**的两个端点。Mean Field 理论提供了连接两者的桥梁。

18.1 从粒子系统的角度看神经网络

回顾两层 μP 网络 $f(x) = \frac{1}{m}\sum_{j=1}^{m} a_j\sigma(w_j^\top x)$。把每个神经元 j 看作一个“粒子”，状态为 $\xi_j = (a_j, w_j) \in \mathbb{R}^{d+1}$。网络就是 m 个粒子的**加权投票**。

训练 = 移动粒子。梯度下降让每个粒子 ξ_j 沿着梯度方向走一小步：

$$\dot{\xi}_j = -\nabla_{\xi_j} L = -(f(x) - y) \cdot \frac{1}{m}\nabla_{\xi_j}[a_j\sigma(w_j^\top x)].$$

关键观察：每个粒子的运动方程依赖**所有其他粒子**（通过 $f(x) = \frac{1}{m}\sum_k a_k\sigma(w_k^\top x)$）。这是一个**相互作用的粒子系统**——和物理中的 N-body 问题一模一样。

18.2 $m \to \infty$：从粒子到分布

当 $m \to \infty$，离散的粒子集 $\{\xi_1, \ldots, \xi_m\}$ 用一个**连续分布** μ_t 描述：

$$\hat{\mu}_m = \frac{1}{m}\sum_{j=1}^{m} \delta_{\xi_j} \xrightarrow{m\to\infty} \mu_t \quad \text{（弱收敛）}.$$

网络输出变为对分布的积分：$f(x) = \int a\sigma(w^\top x)\, d\mu_t(a, w)$。

训练动力学变为**粒子分布的演化 PDE**：

$$\boxed{\partial_t \mu_t = \nabla \cdot \left(\mu_t \nabla \frac{\delta L}{\delta \mu}\right).}$$

这就是 **Wasserstein 梯度流**（Mei, Montanari & Nguyen, 2018; Chizat & Bach, 2018）——在概率分布的空间上做梯度下降。$\frac{\delta L}{\delta \mu}$ 是损失对分布的变分导数。

关键直觉——Mean Field 的物理图像

NTK 视角：关注**投票结果**（n 个训练点上的输出）怎么变——函数空间的 ODE。

Mean Field 视角：关注**粒子群的队形**（神经元参数的分布）怎么变——分布空间的 PDE。

两者看的是同一个训练过程，只是描述层次不同。

18.3 三种极限的关系

	NTK 极限	**μP 极限**	**Mean Field 极限**
适用架构	任意	任意	**仅两层**
极限动力学	线性 ODE	非线性 ODE	Wasserstein 梯度流 PDE
特征学习	无	有	有
全局收敛	有（线性收敛）	更难证明	有（某些条件下）
数学工具	线性代数、核方法	Tensor Programs	最优传输、PDE

关键关系：

- 对两层网络，μP 极限和 Mean Field 极限**一致**——都描述了特征的非平凡演化。
- NTK 极限是 Mean Field 极限的**线性化近似**：当粒子分布只有 $O(1/\sqrt{m})$ 的小扰动时，Wasserstein 梯度流退化为线性核回归。
- Mean Field 的优势：对两层网络能给出**全局收敛的严格证明**。
- Mean Field 的劣势：**只能处理两层网络**。多层的 Mean Field 理论是开放问题。μP 通过 Tensor Programs 可处理任意架构。

18.4 Wasserstein 距离与特征学习的量化

Mean Field 理论给了**精确量化** feature learning 的工具：Wasserstein-2 距离 $W_2(\mu_0, \mu_T)$ 度量 "把初始队形重组成最终队形，平均每个粒子要走多远"。

	NTK regime	μP / Mean Field
每个粒子的位移 $\lvert\Delta\xi_j\rvert$	$O(1/\sqrt{m})$	$O(1/m)$
$W_2(\mu_0, \mu_T)$	$O(1/\sqrt{m}) \to 0$	可达 $O(1)$（取决于梯度的协调结构）
解读	队形没变（特征冻结）	队形可发生质变（特征学习）

为什么 μP 下 W_2 可以是 $O(1)$，尽管每个粒子只动了 $O(1/m)$？

关键在于 m 个粒子的位移是**协调的**（由梯度驱动，方向依赖于训练数据），而不是独立随机的。如果 m 个粒子各自独立随机走 $O(1/m)$ 步，W_2 仍然 $\to 0$；但如果它们的走向有**系统性结构**（例如正参数的粒子向右、负参数的向左），分布的形状就会发生 $O(1)$ 的变化。

类比：一千个人各走一小步。如果方向随机，队形不变；如果左边的人都向右、右边的人都向左，队形从 "两端聚集" 变成 "中间聚集"——每人走的不远，但分布彻底变了。

μP 保证了这种协调结构的**存在**（因为梯度有数据驱动的方向），但 W_2 的具体值取决于任务和数据——它不是自动 $O(1)$ 的，而是 "可以达到 $O(1)$" 的。

18.5 NTK 作为 Mean Field 的线性化

这个关系可以精确写出。设 $\mu_t = \mu_0 + \epsilon\,\nu_t$ 是初始分布的小扰动（$\epsilon \ll 1$ 对应 NTK 的 $1/\sqrt{m}$）。将 Wasserstein 梯度流在 μ_0 处做一阶 Taylor 展开：

$$\partial_t \nu_t = -\mathcal{L}_{\mu_0}\,\nu_t + O(\epsilon),$$

其中 $\mathcal{L}_{\mu_0}$ 是一个**线性算子**，恰好由初始 NTK Θ^* 确定。忽略 $O(\epsilon)$ 就回到了 NTK 的线性 ODE $\dot{\mathbf{r}} = -\mathbf{\Theta}^*\mathbf{r}$。

所以：NTK 动力学是 Mean Field 梯度流在初始分布附近的线性化极限。（注意这不是严格的微扰展开——NTK 极限和 Mean Field 极限的取法不同，前者 $m \to \infty$ 后核冻结，后者 $m \to \infty$ 后分布演化。两者的关系是：如果在 Mean Field 极限中额外要求分布变化很小，就回到了 NTK。）当分布变化超出线性化范围（μP regime），非线性项不可忽略，NTK 近似失效，必须用完整的 Wasserstein 梯度流。

★ 手算 11：三种 regime 的数值对比（思想实验，5 分钟）

$m = 1000$ 个粒子，初始均匀分布在 $[-1, 1]$ 上。训练后：

NTK regime： 每个粒子走 $O(1/\sqrt{m}) \approx 0.03$。$W_2 \approx 0.03$。分布几乎不变（还是均匀的，只是整体微微平移）。

μP regime： 每个粒子走 $O(1/m) = 0.001$。但走向有结构：$\xi_j > 0$ 的粒子向右，$\xi_j < 0$ 的向左。分布从 Uniform$[-1, 1]$ 变成了“中间凹、两端凸”的形状。W_2 远大于单个粒子位移——具体值取决于梯度结构，但关键是 $W_2 \not\to 0$。

发散 regime（$a + c < 1/2$）： 每个粒子走 $O(1)$ 甚至更大。分布完全重组。但输出 f 也发散了——不是合法的训练。

要点： W_2 能看到单个粒子位移看不到的“集体效应”。μP 的精妙之处在于：每个粒子只动了 $O(1/m)$（保证极限存在），但 m 个粒子的**协调运动**可以产生 $O(1)$ 量级的分布重组——具体能否实现取决于任务，但 μP **不排除**这种可能（而 NTK 参数化下 $W_2 \to 0$，feature learning 被参数化本身**禁止**了）。

19 本周总结

三句话总结

1. NTK 理论把 infinite-width 训练动力学精确映射为线性 ODE，给出核回归闭式解和全局收敛保证，但代价是特征冻结（无 feature learning），且最优超参数随宽度漂移。

2. μP 通过联合设计输出层参数化（W_{out} 方差 $\propto 1/d^2$ 而非 $1/d$）和逐层学习率，使得前向/反向信号稳定、损失增量 $\Delta L = \Theta(1)$、且特征变化 ΔY_k 不退化——从而在 infinite-width 极限下保留 feature learning 并支持超参数迁移。

3. 超参数迁移是 μP 的杀手级应用：在小代理模型上搜索最优超参数，直接用于大模型，无需额外调优。

参考文献

[1] Jacot, A., Gabriel, F., & Hongler, C. (2018). Neural Tangent Kernel: Convergence and Generalization in Neural Networks. *NeurIPS*.

[2] Du, S., Zhai, X., Poczos, B., & Singh, A. (2019). Gradient Descent Provably Optimizes Over-parameterized Neural Networks. *ICLR*.

[3] Allen-Zhu, Z., Li, Y., & Song, Z. (2019). A Convergence Theory for Deep Learning via Over-Parameterization. *ICML*.

[4] Lee, J., Xiao, L., Schoenholz, S., et al. (2019). Wide Neural Networks of Any Depth Evolve as Linear Models Under Gradient Descent. *NeurIPS*.

[5] Yang, G. & Hu, E. (2021). Tensor Programs IV: Feature Learning in Infinite-Width Neural Networks. *ICML*.

[6] Yang, G., Hu, E., Babuschkin, I., et al. (2022). Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. *NeurIPS*.

[7] Yang, G. (2019–2023). Tensor Programs I–V. *NeurIPS/ICML/arXiv*.

[8] Mei, S., Montanari, A., & Nguyen, P.-M. (2018). A Mean Field View of the Landscape of Two-layer Neural Networks. *PNAS*.

[9] Chizat, L. & Bach, F. (2018). On the Global Convergence of Gradient Descent for Over-parameterized Models using Optimal Transport. *NeurIPS*.

[10] Rahaman, N., et al. (2019). On the Spectral Bias of Neural Networks. *ICML*.

[11] Dey, N., et al. (2023). Cerebras-GPT: Open Compute-Optimal Language Models Trained on the Cerebras Wafer-Scale Cluster. *arXiv:2304.03208*.

[12] Bordelon, B. & Pehlevan, C. (2022). Self-Consistent Dynamical Field Theory of Kernel Evolution in Wide Neural Networks. *NeurIPS*.

[13] Zheng, C., et al. (2025). Scaling Diffusion Transformers Efficiently via μP. *NeurIPS 2025*.

[14] Blake, C., et al. (2024). u-μP: The Unit-Scaled Maximal Update Parametrization. *arXiv:2407.17465*.

[15] Dey, N., et al. (2025). Completed Hyperparameter Transfer across Modules, Width, Depth, Batch & Duration. *arXiv:2512.22382*.